

分类号_____

学号 M201471739

学校代码 10487

密级_____

华中科技大学

硕士学位论文

基于 NodeJS 与云存储的文件管理平台 的设计与实现

学位申请人：李大松

学科专业：电子与通信工程

指导教师：夏天 教授

答辩日期：2015.05.20

**A Thesis Submitted in Partial Fulfillment of the Requirements
for the Degree of Master of Engineering**

**Design and Implementation of Document
Management Platform Based on NodeJS and
Cloud Storage**

Candidate : Li Dasong
Major : Electronic Information&Communications
Supervisor : Prof. Xia Tian

Huazhong University of Science and Technology

Wuhan, Hubei 430074, P. R. China

May 2015

摘要

随着网络技术的迅猛发展，网络已然成为各类文件流通的管道。而文件作为网络传输的基本单元，如何妥善地保存与管理是当下亟待研究的内容。在传统文件管理方式中，直接将文件存储在本地进行管理，而基于网络技术的云存储，依赖网络中各类互连的存储设备，更加有效地实现数据的在线存储，提高了文件管理的便捷性、安全性、可靠性。

本文旨在利用 Web 相关技术设计并实现一个基于 NodeJS 与云存储的文件管理平台。首先，平台采用(Linux+NodeJS+Express+MySQL)架构实现一个易于扩展、轻量的 Web 服务器；其次，利用云存储服务提供的文件存储以及管理功能，结合 Web 前端以及 Web 服务器实现文件的上传、删除等处理；再者，利用 Web 技术实现文件管理平台上的各类功能：包括文件内容的在线分享，多种类型文件的在线预览与播放，平台上用户之间的交流互动，以及社区资源互助等功能；同时利用 HTML5 WebSocket 协议建立一个全双工的实时消息通信信道，完成平台上各类消息的实时分发与用户通知。最后将平台部署到阿里云服务器，对线上的文件管理平台进行系统化的功能测试以及性能测试。

在最后的实现中，文件管理平台的各项指标都达到预期目标，并且通过了功能与性能测试。使用 NodeJS 搭建了高性能、高可用性的 Web 服务器；利用云存储低成本、高效、易于管理的巨大优势，安全、有效地管理在线文件；而且通过 Web 前端技术实现了各类文件的预览。

关键字：文件管理 NodeJS 云存储 WebSocket

Abstract

With the rapid development of network technology, network has become a pipe of files distribution. File as a basic unit of the network transmission, saving and managing appropriately is the focus of this paper to research. In the traditional way, files are stored and managed locally. But cloud storage relied on the network interconnection of storage equipment, can more effectively achieve the online storage and improve file managing convenience, security, and reliability.

The purpose of this paper is to design and implement a document management platform based on NodeJS and cloud storage. First of all, the platform utilizes (Linux + NodeJS + Express + MySQL) architecture to realize an extensible, lightweight Web server; Secondly, implement file uploading, deleting and other functions with web technology and the advantage of file storage and management service of cloud storage; Furthermore, achieve various functional management, including online file sharing, documents previewing, multimedia playing, users interacting, as well as community resources sharing etc. Moreover, employ HTML5 WebSocket protocol to establish a full-duplex, real-time messaging channel, and then complete various types of messages and user notifications distribution. Finally, the platform will be deployed to AliCloud Elastic Compute Service. And test the functions and performance of online platform systematically.

In conclusion, all targets are achieved and conform to desired goals after a comprehensive test. A high-performance, high-availability Web server is built with NodeJS. Online files are managed effectively with the low-cost, high-performance, manageable benefits of cloud storage. Different types of files can be previewed online with the Web front-end technology.

Keywords: Document management NodeJS Cloud storage WebSocket

目 录

摘 要.....	I
Abstract.....	II
1 绪论	
1.1 研究背景及意义	(1)
1.2 国内外研究现状	(2)
1.3 研究目标与内容	(3)
1.4 论文组织结构	(4)
2 相关技术背景介绍	
2.1 平台技术架构	(6)
2.2 云存储技术	(10)
2.3 Express 框架.....	(10)
2.4 Web 前端技术	(12)
2.5 本章小结	(15)
3 平台的需求分析与设计	
3.1 需求分析	(16)
3.2 平台框架	(19)
3.3 功能模块设计	(20)
3.4 数据库设计	(26)
3.5 本章小结	(30)

4 文件管理平台的实现与测试

4.1 文件管理模块(31)

4.2 基于 WebSocket 的社交互动模块(40)

4.3 基于社区的资源互助(45)

4.4 平台测试(48)

4.5 本章小结(53)

5 总结与展望

5.1 论文工作总结(54)

5.2 未来工作展望(55)

致谢.....(56)

参考文献.....(57)

1 绪论

1.1 研究背景及意义

互联网急速的发展过程中,网络数据量在不断扩大,2013 年中国就产生了 0.8ZB 的数据总量,预计 2020 年,将是 2013 年的 10 倍。本地保存的文件也在增大,而且本地各类文件的预览都需要对应的预览软件。当文件在不断增多、文件格式不断更新的当下,如何更加有效、安全的保存、预览文件成为不得不考虑的问题。而传统的保存、预览方式都是将文件内容拷贝到本地,然后再在本地使用相应的软件预览文件。但是在今天,这种管理方式的不足之处在不断显现:

1) 安全性存在隐患

本地的数据都是存放在各个分区的磁盘空间中,由于设备是有一定寿命的,而且设备是会时常发生故障的,虽然可以使用多设备备份,但是这种拷贝复制的方式极其不便捷,而且无法设备何时出现故障,并在故障时间前备份文件。一旦设备发生故障,设备上存储的数据极有可能丢失,而且难以恢复。

2) 跨设备访问成为瓶颈

在人们活动地点频繁变动的当下,我们不可能到达每个地方都携带自己的设备,有些设备根本不方便携带;而且因为设备的种类丰富,当文件需要传递到其他设备时,由于各设备之间的区别,还需要使用特定的接口进行互联。因此当在其他地方需要使用设备上的文件时,传统上,需要使用一个外部设备作为中转,将需要的文件拷贝到另一台设备上,然后再使用本地软件预览。然而由于编码等方式的不同,可能在一台设备上可以正常预览的文件,其他台设备却无法显示,造成极大不便。

3) 预览方式的复杂性

本地的文件在预览时,都需要有对应的预览软件。比如 word 文档需要 Office 或者 WPS 等软件浏览,音频文件需要播放器软件进行播放,视频文件的预览又要视频播放器,使得预览文件不便。

这些传统的文件保存、预览、跨设备使用出现的问题成为了极大的桎梏,而在互联网技术迅速变革的当下,利用 Web 前端、云存储等技术可以比较完美的解决这

些出现的问题,实现一站存储,随处访问的优势。而且在 Web 前端技术发展迅速的今天,在浏览器端可以直接利用 HTML5^[1]提供的应用程序接口播放音频与视频文件。而云存储提供的文件存储以及数据处理功能结合前端技术,可以无缝实现多种文档的在线预览。这种在线文件管理平台的构建完美解决了跨设备访问文件的瓶颈,简化了文件预览的复杂度。还可以极大限度的提高文件安全。

1.2 国内外研究现状

国内外在云存储空间上的产品可谓是百花齐放。国外比较知名的有亚马逊云存储^[2](Amazon Cloud Drive)、谷歌硬盘(Google Drive)^[3]、微软 OneDrive 云存储^[4]等。国内的文件存储平台有 360 网盘、百度云盘、腾讯微云等等,为用户提供了简捷易用的云上文件管理方式。根据上述列出的国内外文件管理平台,下面分别进行详细介绍。

1) 亚马逊云存储

亚马逊作为云计算的引领者,在各个方面都有很优秀的产品,其中就包括云存储,亚马逊 Cloud Drive 云端存储服务。Cloud Drive 提供一站存储式服务,一次存储,任何联网设备上都能够访问云端文件。Cloud Drive 利用云端存储的优越性,保证文件在本地文件损毁或者丢失的情况下,可以方便地实现本地文件恢复。

2) 谷歌硬盘

Google Drive 同样也提供在云端硬盘上的图片、文件、影片等文件的存储,而且云端的文件可与联络人分享,并可以在线预览存储的各类文件。同时 Google Drive 还能够在线创建与编辑文件。

3) 微软 OneDrive 云存储

OneDrive 提供了一定的免费空间,用户可以将文件存储在这个免费空间中,其拥有基本的文件管理方式,默认情况下有图片、文档、文件夹。同时利用 OneDrive 在线办公功能,可以在线实时创建、编辑文件。

4) 腾讯微云

腾讯微云是近年才开始进入用户眼帘的在线文件存储应用,利用微云可以将本地的文件安全快捷地存储到云端,微云还提供部分文件的在线预览功能,方便查看

存储在线上文件。微云还提供了在线笔记功能，用户利用云笔记可以在线编辑线上的笔记内容。

5) 360 网盘

360 网盘是一款很早在国内使用的文件管理平台，利用免费的存储空间供用户存储以及在线预览文档、图片、音视频等各类文件；而且 360 网盘还实现跨平台的文件同步功能，各类终端都可以利用 360 网盘将文件内容同步到云端服务器，同时可以对云端的文件进行批量下载。

6) 百度云盘

百度云盘作为后起之秀，在文件管理上，体验极致，它除了在线存储、文件同步外，还包括社交、文件分享等功能，让云盘的功能不仅仅局限在了存储方面。

结合了这些中外目前云端文件管理方面的研究，云端存储不在只是局限在文件的存储，还包括多类型文件的在线预览、文件同步、用户社交等功能。但是在文件的在线预览功能上，不管是百度云盘还是 360 网盘都采用第三方插件的形式。但是在前端技术越发成熟的今天，可以直接利用前端技术来实现文件的在线预览。而且在用户社交功能上，大多数的云端文件存储平台上都没有完善，这个也是本文的重点研究部分。最后，各大存储管理平台上都缺乏用户自主请求资源功能，本文也对这部分进行了探索。

1.3 研究目标与内容

本文主要旨在研究，在 NodeJS^[5]平台上，利用基于 MVC^[6]框架的 Express^[7]组件库，构建一个 Web 服务器；探究云存储的基础服务，并在项目开发中应用云存储提供的存储与数据处理服务；以及利用 Web 前端技术实现一个基于 NodeJS 与云存储的文件管理平台。

如何利用 NodeJS 平台构建一个高性能、高可用性的 Web 服务器是本文研究的重点。根据云存储服务的优点为平台提供数据存储量以及数据处理功能，是本文研究的一个要点，最后设计实现具有可用性、实用价值高的文件管理平台是本文的目标。

本文的具体研究工作如下：

1) 构建一个基于 NodeJS 的 Web 服务器

利用 NodeJS 的优势,可以方便快速地开发网络服务以及搭建应用平台^[8]。Express 组件采用 MVC 设计模式构建平台的总体结构,组织服务器端的各个层级,在控制层处理前端与后端的 HTTP^[9]请求,视图层使用 EJS 模板引擎,动态生成 HTML^[10]文档,模型层主要管理平台各类数据,并按要求给控制提供数据支撑。

2) 设计基于云存储的文件管理平台的架构

本文着眼于文件管理,因此需要构建文件存储的架构。浏览器端的文件上传、下载功能需要与云存储服务器的协同操作,管理存储在云上的文件需要与服务器协同交互。如何构建平台的整体结构框架,提升平台的访问性能、稳定性能是本文的重点研究内容。

3) 实现文件管理平台的前端逻辑以及后端业务模块的开发

文件管理平台在框架结构上包括以下三部分:

(1) Web 前端

实现各类 Web 页面显示,各种前端逻辑的交互,文件预览以及前后端数据操作。

(2) 云存储服务

平台上的所有文件资源都存储到云存储服务器分配的存储空间中,前端根据需从云存储服务器中获取指定的文件内容;前端的文件上传、删除等操作需要同 Web 服务器与云存储服务器的协同交互,通过云存储提供的处理接口对文件资源实现增删功能。

(3) Web 服务器

后端服务器包括三个层级,服务器的控制层主要为 Web 前端提供数据以及逻辑交互接口;在数据模型层,服务器需要与数据库管理系统进行数据的交互,视图层需要将数据结合渲染模板将 HTML 文档发送给前端,进行显示。

1.4 论文组织结构

本文总共分为五个章节,每章的详细介绍如下:

第一章绪论 介绍文件管理平台的研究背景与意义,以及当前文件管理平台的发展现状、研究目标与内容。

第二章相关技术背景介绍 分别介绍本文所采用的架构形式，平台中所要使用到的核心技术知识，包括云存储技术、NodeJS 服务器平台、Web 前端技术等。

第三章平台需求分析与设计 根据用户需求与功能需求分析，划分平台需要的功能模块，并完成各模块的功能设计，本章最后完成基于 MySQL^[11]的数据库设计。

第四章平台的各模块实现与测试 根据第三章的需求分析，详细介绍部分功能模块的实现思路、方法与细节；最后对平台的各个模块的功能与性能进行测试。

第五章平台的总结与展望 综合描述对已经完成工作的总结和对平台未来发展提出的建议与展望。

2 相关技术背景介绍

本文利用 NodeJS 平台、云存储服务以及 Web 前端技术对文件管理平台进行设计与实现。技术是支撑平台构建的基础，在本章节中，将对平台中利用到的核心技术背景和相关知识做一个简单的介绍，了解相应的技术发展和特点，并对具体应用到的技术进行描述与分析。

2.1 平台技术架构

本节将重点介绍文件管理平台所采用的技术架构。平台采用 Express 组件实现后端的各类功能逻辑，数据库管理系统采用 MySQL，在数据库之上加一层 Redis^[12]缓存层，方便数据的存储与管理，增强前端数据获取的高效性。Web 前端的模块加载器使用 RequireJS，使用 CSS 的预处理工具——LESS，并且结合 Bootstrap 进行前端显示页面的渲染，利用七牛云存储^[13]服务器管理各类文件资源。相关技术知识层次图如 2-1 所示：

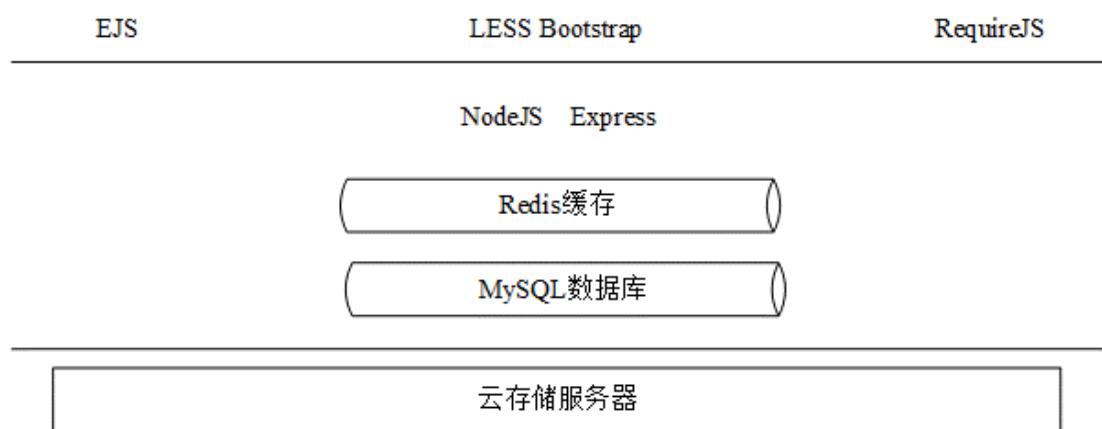


图 2-1 平台技术层次图

2.1.1 NodeJS 平台

NodeJS 是基于 Google Chrome V8^[14]引擎的 JavaScript 执行平台，可以高效地处理服务器端的 I/O 逻辑交互。而且在 V8 引擎的支持下，JavaScript 可以友好地进行后端功能开发，与开发前端逻辑共用一种语言，并且使用同一类渲染模板，大大提升了项目效率，减少了平台在开发过程中的复杂度。NodeJS 最初版本在 2009 年发布，

但是由于其优秀的功能，截止本文完成前，NodeJS 最新稳定版本已经到了 5.4.1，而且 NodeJS 有自己的包管理生态系统，包含了丰富的开源工具包，各种工具包的使用非常便捷。在开发过程中只需利用 NodeJS 集成的包管理工具下载所需要的开发包，使用开发包提供的功能接口，辅助平台功能模块的开发。NodeJS 的迅猛发展离不开他的诸多优点：

1) 单线程

单线程主要负责顺序执行程序逻辑，减轻了程序处理过程中状态同步，上下文切换过程中的开销^[5]。NodeJS 处理一个异步任务时，会将此异步任务丢到任务队列中，只有执行栈的所有任务执行完毕后，才会去读取任务队列的异步任务进行处理。

2) 非阻塞 I/O

NodeJS 在调用 I/O 接口时，系统调用高性能的 I/O 事件处理库 libuv。在处理 I/O 请求时，底层调用 libuv 创建一个新的线程处理这次的异步任务，处理完毕后，再将结果回传给调用线程。

3) 事件驱动

在程序执行过程中，libuv 会利用其核心函数 `uv_run` 不断检查任务队列中是否有就绪状态的事件，如果存在已经就绪的事件，就会进行触发，执行一个回调函数。

4) RESTful API

REST(Representational State Transfer)是一种针对网络应用的设计与开发方式^[15]。Express 组件正是利用 REST 机制，通过识别客户端的 HTTP 的标准请求方法，判断资源的处理方式，比如 GET 请求代表资源的获取，PUT 请求表示更新资源，POST 请求指示创建资源，DELETE 请求表示删除资源，REST 方式应用到实际项目开发中前后端开发过程中的不必要协调，而且增强项目的可扩展性。

NodeJS 在解析代码过程中，通过代码中的异步任务，调用 NodeJS 的应用接口，底层的 libuv 将所有异步任务交给新创建的进程处理，异步任务一旦执行完成，执行结果就会返回给任务队列，libuv 的核心函数 `uv_run` 不停地检查事件队列，通过事件循环将完成的异步任务结果交给主线程，最后再将结果返回给用户。NodeJS 平台内部运行结构图如 2-2 所示：

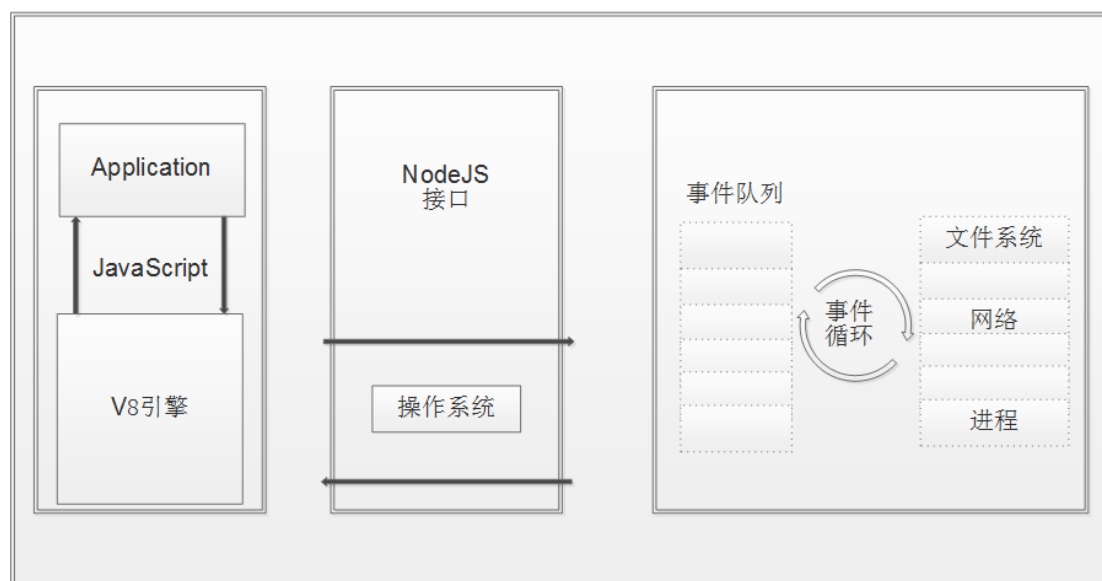


图 2-2 NodeJS 运行机制图

2.1.2 Redis

Redis 是一个基于 key-value 的高性能非关系型数据库^[16]。因为 Redis 的灵活多变的数据结构和数据操作的优势，它在构建实时消息系统、实现数据缓存方面得到广泛应用。Redis 包括以下特点：

Redis 包含多种数据类型：包括字符串、集合、链表以及哈希^[17]。根据需求，使用不同的数据类型存取数据，可以方便的使用 Redis 提供的数据类型，极大简化单一数据类型在存储过程中的处理开销。

Redis 事务控制：Redis 只能保证一个连接端发起的事务中的命令可以连续的执行，而中间不会插入其他连接的命令^[17]。一般情况下，Redis 会直接处理连接发送来的命令，但是如果发出的命令是 multi，连接就会进入一个事务上下文，这个连接的后续命令会被放到一个队列中，等待连接接收到 exec 执行命令后，Redis 才会顺序执行队列中的命令，并将所有的运行结果打包发送给连接端，最后结束事务上下文。

Redis 数据持久化：Redis 拥有两种不同的持久化的方案：快照与 AOF(Append Only Mode)^[17]。快照方式是基于时间点的持久化方式，适合于备份场景，通过设置快照时间间隔，在间隔期间，RDB 文件临时存储当前改变的数据，存储过程完成后，再用新的文件替换旧文件。而 AOF 会将每个写命令都会记录下来，即使宕机，也能

够将最新的数据恢复出来。因此在系统停止时，快照方式最后写入的 Redis 数据就会全部丢失，而 AOF 方式的 `fsync every second` 在最坏的情况下也只可能丢失一秒钟的数据，所以 AOF 相对快照更加可靠。

Redis 支持主从模式：Redis 支持将一个 Redis Server 作为另一个 Redis Server 的从机，主机只要定期向从机提供数据。而且从机同样可以设置为一个 Redis Server 的主机，如此形成了一个 Redis server 集群。

Redis 支持自定义配置：Redis 服务器默认端口使用 6379，配置快照方式进行数据存储时，使用 `save <seconds> <changes>` 命令，其中 `second` 表示数据同步的时间间隔，`changes` 表示时间间隔内的数据更改次数。在指定设置的时间间隔内，如果更改次数大于或等于设置的次数，将会执行数据更新。AOF 的配置方式更加简单，执行 `appendonly <use>` 命令，如果命令中的 `use` 为 `yes` 则表示使用 AOF 方式实现数据的持久化。

2.1.3 MySQL

MySQL 作为数据库管理系统，因其易于使用、精巧、开源等特性，在各类中小型网站开发中倍受青睐。MySQL 支持多种平台，包括主流的操作系统 Windows、Linux、Unix 等，并且 MySQL 系统提供了面向多种编程语言的编程接口，包括 C、C++、PHP、NodeJS 等，方便用户使用不同的处理方式管理数据库系统中的数据。因为 MySQL 源代码开放，在不同的需求下，可以配置一个适用的个性化数据库管理系统。MySQL 作为关系型数据库，在使用 SQL 结构化查询语言进行数据的查询、插入、删除和修改等操作也带来了极大的便捷。在创建数据库时，可以利用 MySQL 可视化客户端 Navicat for MySQL^[18]，可视化的数据表结构简单明晰，更加适合管理数据库的各类数据。综合以上，由于 MySQL 的小巧、灵便、快捷等优点，在构建中小型网站应用中，是一个完美的选择。

2.2 云存储技术

云存储是在云计算基础上延伸而来的产物^[19]，云存储就是将网络中的各类存储设备进行互连，为用户提供在线的数据存储以及业务访问的存储系统。云存储在结构上分为四层：数据存储层、基础管理层、应用接口层以及用户访问层^[21]。

基础管理层作为云存储的核心层级，通过集群、分布式等技术，完成系统存储设备之间的协同工作，提供更大、更强、更好的数据访问性能^[22]。访问层提供给用户一个访问云存储系统的一个入口，只要是通过授权的使用者，就可以在认证共过后进入云存储系统，实现与其他平台的对接，使用其功能服务。

云存储在各个领域中的广泛应用，相对传统存储方式，依赖于其成本低廉、成效迅速的巨大优势。在云存储方式下，企业不需要投入额外的资金搭建平台^[20]，这种方式极大减轻了企业的负担。云存储的存储方式灵活，云存储在使用时可以根据业务的变化、需求的更新可以更改云储存提供的使用方式，实现按需使用。

2.3 Express 框架

2.3.1 框架简介

Express 是轻量灵活的 NodeJS Web 应用框架^[23]。框架中的中间件处理方式，可以在一个请求过程中，任意排列组合需要使用到的方法。而且 Express 提供了方便使用的模块化管理模式，在构建 Web 应用时更加便捷。

1) 路由

路由指的是一条客户端到服务器端的请求，Web 服务器根据客户端 HTTP 请求的路径以及方法信息，匹配到特定的路由处理函数，最后将执行结果返回给客户端。一条路由请求由路由方法、路由路径、路由句柄组成。路由方法指定了一条请求的请求方式，例如 `get` 请求方法会对应一个获取资源的路由方法。路由路径定义请求从起点到终点的一条路径，确定路由的方向。路由句柄是一个或者一组处理方法，路由句柄负责在路由请求匹配时，调用这个句柄的执行结果应答客户端。

2) 中间件

中间件是一个函数，主要功能包括修改请求和响应对象、终结请求响应循环以及调用堆栈中的下一个中间件。

3) 模板引擎

模板引擎实现用户界面与业务数据分离的一种方式。模板引擎会加载模板文件，然后结合服务器端的数据，解析出 HTML 文档或者片段，然后将生成的片段或者文档发送给客户端，进而实现页面的显示。

4) 错误处理

Express 框架可以根据需求，自定义错误处理方式。当在业务处理过程出现错误时，会根据不同的错误类型，使用不同的错误响应方式。Express 还内置了一个错误处理句柄，可以捕获任何可能出现的错误，并将错误的堆栈追踪信息显示在控制台端。

2.3.2 MVC 模式

MVC 是 Model-View-Controller 的缩写。MVC 模式将数据处理、视图显示与逻辑控制各部分分离开来，三者互相独立又相互依存，三者独立处理自己的业务逻辑，三者又彼此关联共同实现系统的完整功能。

MVC 模式由三个部分组成：模型、视图、控制器^[24]。模型是数据处理中心，主要负责应用的数据封装、状态管理；视图是最接近用户的部分，主要承担页面的渲染以及处理用户交互；控制器是模型与视图之间的信息传递通道，委派模型获取或处理数据，再将数据内容传送给视图进行渲染。

MVC 这种三层分离的构架模式，在应用程序设计开发上具有很多好处^[25]。MVC 可以达到尽可能的功能复用，在 MVC 模式中，一个模型可以应对多个视图，模型只负责数据的处理，不去耦合视图的显示，这样不管使用何种方式显示视图，模型都只负责数据的处理。而且在 MVC 模式中，视图的修改不会影响模型的数据处理，同时视图的数据需求变化，只需要在模型层更改对应的数据管理模块，极大提高了项目的可扩展性。其中 MVC 结构模式如图 2-3 所示：

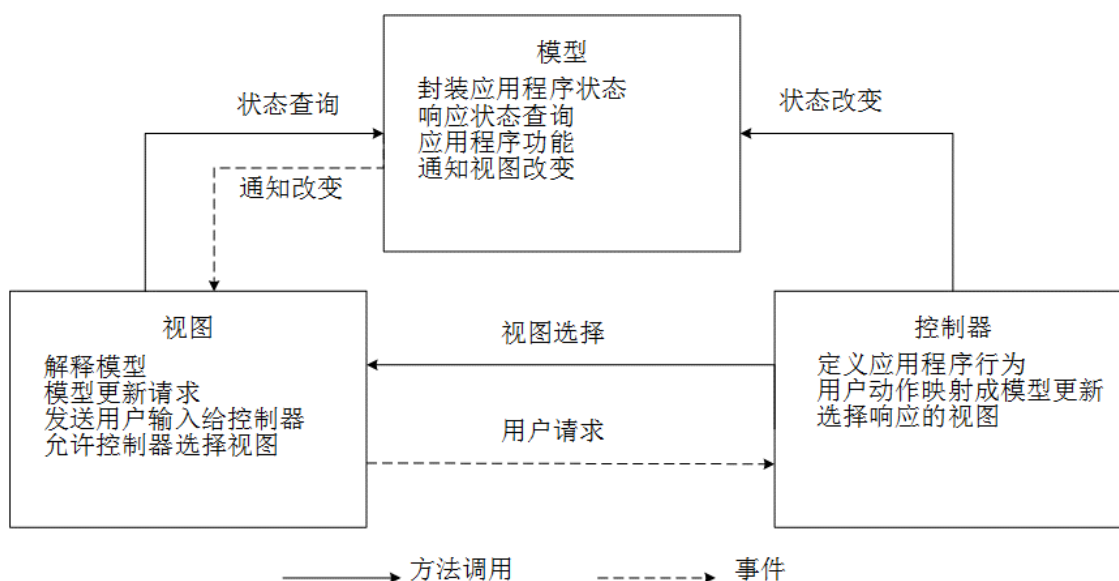


图 2-3 MVC 模式结构图

2.4 Web 前端技术

2.4.1 Web 标准

Web 标准是一系列标准的集合，主要分为三个部分：结构、表现和行为^[26]。与之对应的技术标准也包括三个部分^[27]。

1) 结构标准

HTML 是用于描述网页文档的一种标记语言。作为 Web 的基础之一，在 HTML 之后，包含文本、图像、视频等各种信息的页面逐渐出现在人们的视野中。XML(Extensible Markup Language)是标准通用标记语言(SGML)的子集^[28]，它剥离了数据本身和数据的表现，也适当的分离了数据的值和语义，为分布式或异构的数据交换给予极大的帮助。

2) 表现标准

CSS(Cascading Style Sheets)是一种提升文档表现形式的计算机语言^[29]。通过定制 CSS 在网页中的内容，可以改善显示效果，美化网站内容。将 CSS 与 HTML/XHTML 结合，实现网页结构与表现层的分离，更加易于维护网站结构。

3) 行为标准

ECMAScript 是 JavaScript 语言的核心^[30]，是对实现 JavaScript 语言标准各个方面内容的描述，并规定了语言的各个组成部分：语法、类型、语句、关键字、保留字、操作符、对象。

DOM(Document Object Model)是前端操作 HTML 的应用程序编程接口^[31]。DOM 将整个 HTML 或 XML 页面映射成由节点组成的一棵文档树，利用 DOM 提供的 API，可以方便的操作页面的结构和内容，实现对文档节点的增、删、改、查等操作。

2.4.2 Ajax

Ajax(Asynchronous JavaScript And XML)^[32]，是结合了多项技术的总称，包括了 JavaScript、XMLHttpRequest、XML 和 CSS。为了实现客户端异步无刷新操作，Ajax 技术在客户端和服务端之间加了一个虚拟的中间层。传统的 Web 应用在页面内容需要变化时，需要从服务器获取完整的页面^[33]。而在利用 Ajax 开发的应用中，客户端只需要向服务器请求页面需要更新的内容，从而实现部分刷新。这种方式极大地节约了网络传输带宽，用户交互也更加友好。

传统模式下的 Web 应用都是在客户端使用表单的形式，将表单中要提交给服务器，服务器获取表单并处理后，生成一个新的网页供客户端显示。而相比传统模式，通过 Ajax 引擎，浏览器不直接与服务器通信，发送的请求都要通过 Ajax 引擎处理后，再与服务器进行通信，从而完成异步无刷新的交互模式。其中传统模式与 Ajax 模式的对比图如 2-4 所示：

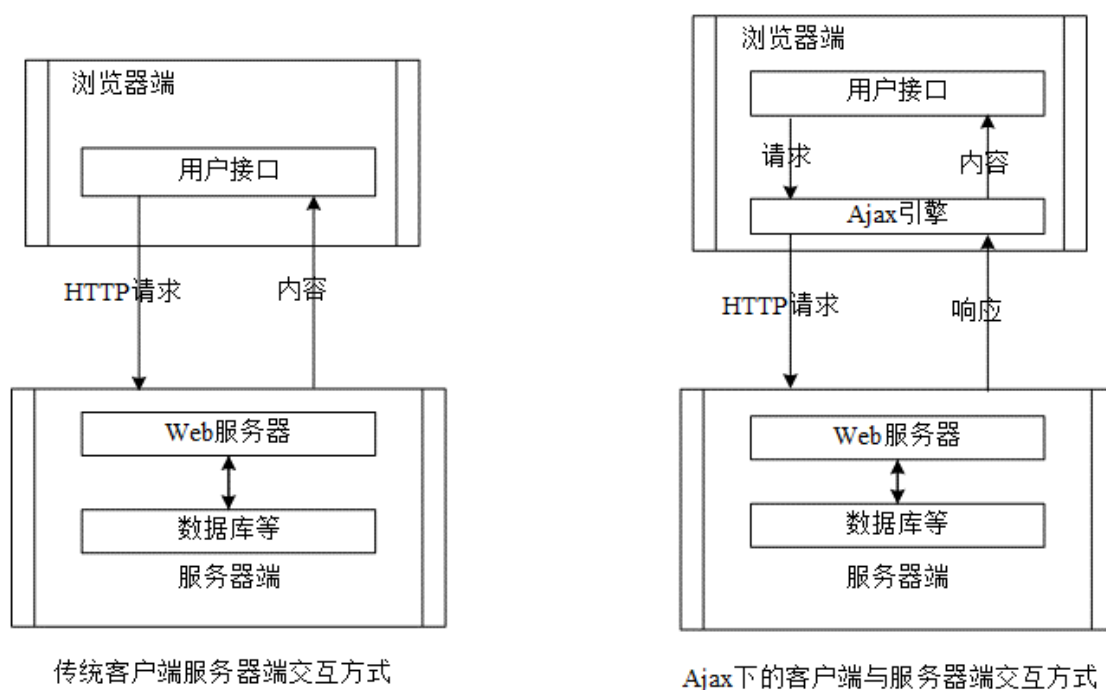


图 2-4 传统交互模式与 Ajax 交互模式对比图

2.4.3 WebSocket

WebSocket^[34]是在 HTML5 中新定义的通信协议。通过 WebSocket 建立的连接，客户端和服务端就可以利用这条连接实时发送和接收数据^[35]。WebSocket 通信协议建立在 TCP 协议之上，通过维持的一条 TCP 连接进行双向的数据交互。这种双向通信的建立分成三个阶段，首先通过 HTTP 协议传送握手数据；服务器根据客户端的握手数据，再向客户端响应一条应答消息；本地拿到服务器的应答消息后，会与本地的 Sec-WebSocket-Key 做一次验证。如果验证通过，表示握手成功，随后就会使用 WS 协议进行数据的实时传递^[36]。

Socket.IO^[37]是根据 WebSocket 协议封装的通信框架，服务器端与 Web 前端可以利用 Socket.IO 框架消除复杂的建立连接操作，只需要注重业务逻辑处理，更加便捷地实现实时通信。

通信建立流程图如图 2-5 所示：

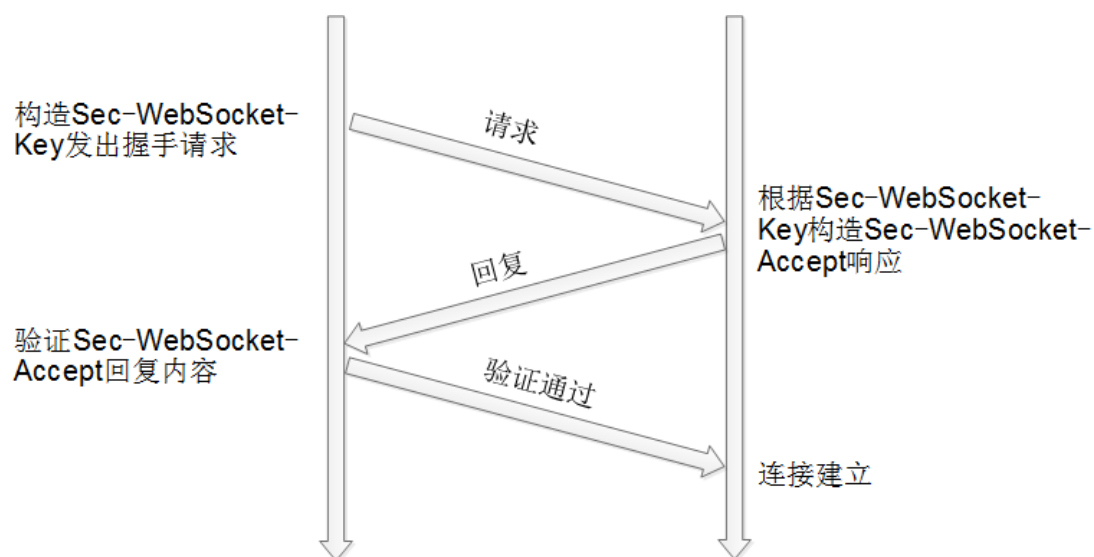


图 2-5 WebSocket 建立流程图

2.5 本章小结

本章通过对文件管理平台设计与实现中需要设计的技术，描述了平台的技术层次图。根据技术层次图，介绍了构建平台框架的技术知识结构，然后分层描述了 Express 构建 Web 服务器的技术背景以及云存储技术和 Web 前端技术等相关知识。

3 平台的需求分析与设计

需求分析是系统设计的罗盘，指导设计的方向。有了符合需求的设计，才能按照设计的功能来进行平台的实现。因此需求分析是平台的支架，支撑着整个平台的功能逻辑，指引着平台功能的实现。本章将首先对平台的用户需求、平台功能需求进行大致分析介绍，然后根据平台的各项需求，对整体框架结构进行规划设计。针对需求分析的结果与平台的整体架构，确定平台的整体功能划分以及各个模块的详细设计方案，再详细设计各功能模块。最后根据整个平台的数据需求，设计文件管理平台的数据结构以及对应的数据库系统。

3.1 需求分析

平台的需求分析包括两大部分：用户需求分析、功能需求分析。用户需求分析是站在使用者的角度上，希望平台能够提供给使用者所关注的各项功能，功能需求分析是站在整个平台的角度上，参考用户的需求，平台所要完成的功能。

3.1.1 用户需求分析

用户是平台设计的指南针，指导着整个平台的结构以及各个模块的功能实现。作为文件管理平台的用户，希望平台提供的功能必须切实地达到指定的需求。作为文件管理平台，文件是平台中基础的组成单元；因此用户在文件管理平台上主要需求包括文件的上传、下载、管理以及在线预览。

1) 文件的上传

平台上的基础单元就是文件，用户所需要平台完成的基础功能就是将存储在本地的各类文件存放到平台上，这样用户无论在何处，只需在线，就能够管理线上存放的文件。所以文件首先就应该出现在平台上，因此文件的上传是用户使用该平台的基本需求，也是平台的核心需求。

2) 文件下载

用户上传的文件都存储在云存储服务中，用户在需要使用保存在平台上的文件时，可以下载平台上的文件到本地。或者下载其他用户在平台上分享的文件内容。

3) 文件管理

上传到平台上的文件，用户还需要能够在线管理所有上传的文件，包括更改文件名称、删除已上传的文件等等。同时还可以将部分内容放在一个特定的文件夹下，方便用户管理上传的同一类型文件。因此用户需求中还包含创建文件夹，并且可以在创建的文件夹中管理其中的各类文件。创建的文件夹还能够进行名称修改以及删除。

4) 文件在线预览

用户在使用文件管理平台时，不仅仅需求在平台中存储各类文件、管理各类文件以及文件夹，最主要的是用户不需要将平台上的文件下载到本地预览，直接使用平台提供的功能在线浏览各类文件，包括文档内容以及多媒体资源的预览。基础文档类文件的浏览，包括文本文件、Word 文件、PDF 文件以及 PPT 文件等各文档类文件的浏览，多媒体文件的在线播放，包括音频与视频文件。

根据用户的需求分析，文件管理平台的用户需求结构如图 3-1 所示：

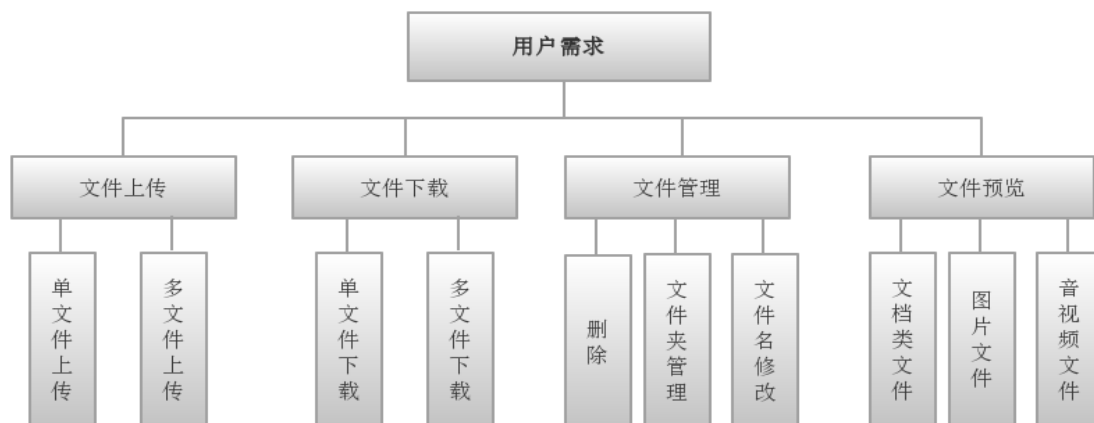


图 3-1 用户需求结构图

3.1.2 功能需求分析

用户的需求最后还是要反映到功能上，用户的种种需求都需要平台提供相对应的功能模块来实现。从功能需求分析，平台功能需求围绕着用户账号管理、文件管理、用户社交信息管理等各个功能模块展开。

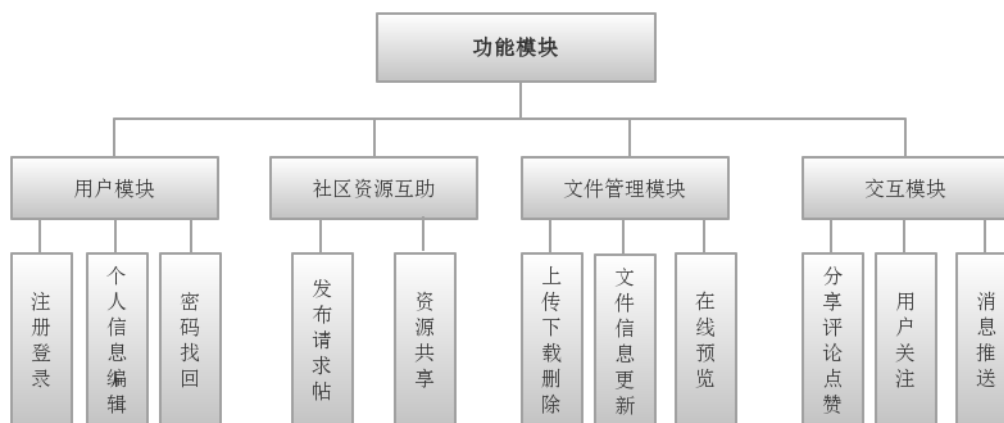
用户账号模块是管理个人信息的基础模块，包括用户名、注册邮箱、个人描述、头像等。用户账号是登录文件管理平台的凭证，在使用平台提供的功能之前，需要使用个人信息登录平台，Web 服务器查询数据库，根据登录信息判断是否已经注册，进而允许或者禁止用户登录平台。只有成功登录平台的用户，才能够使用平台的各类功能。不能成功登录的账号，平台会返回相应的提示信息。平台还要提供找回账号密码功能，用户重置的密码后，再登录平台。用户账号信息还是其他模块的组成部分，比如文件管理模块根据用户的个人信息判断文件的管理权限，社交模块需要用户信息处理各类消息等。

文件管理模块是平台的主要功能模块，包括对平台上的文件管理以及自定义文件夹的管理。文件夹管理包括文件夹的创建、文件夹的删除以及文件夹的重命名；文件夹的管理只需将文件夹信息存放在 Web 服务器的数据库管理系统中，不需要将文件夹信息存储到云存储服务器。而文件的管理包括文件的上传、下载、删除、分享以及在线浏览、播放，因此文件的信息需要存储在 Web 服务器的数据库管理系统中，用户页面的显示，而文件内容需要统一保存到云存储服务器中，当平台需要预览各类文件时，直接从云存储服务获取文件资源在 Web 浏览器端显示。

用户社交模块为用户提供了与线上其他用户进行交流、文件分享的实时互动功能。社交模块包括用户之间的资源互动、消息提示、用户之间的关注与被关注。

其中的资源互动部分包括分享内容的点赞、对分享内容的评论；消息提示包括分享内容评论的消息、用户被关注提示等。

文件管理平台的各功能模块如 3-2 所示：



3.2 平台框架

通过需求分析，文件管理平台的 Web 数据库管理系统需要维护使用者的账号信息、上传的各类文件信息、交互信息；Web 服务器根据客户端的不同请求给予分类处理，包括登录请求、文件上传、Ajax 获取 JSON 数据处理等；云存储为文件内容提供存储与处理功能，用户上传的文件都保存在云存储服务端，而业务服务器的数据库管理系统只存储上传文件的各类信息，云存储服务器还提供了各种文件内容处理功能，比如图片的裁剪、音频文件的格式转换，视频文件切片的处理等。

在整体结构上，文件管理平台基于 B/S 架构，客户端与服务器以请求应答的模式实现数据交互。Web 浏览器主要负责页面的显示，前端逻辑的处理，与 Web 服务器的数据交互，以及与云存储服务器之间的文件内容传输。Web 服务器功能包括，处理前端的各类请求，并根据请求给予客户端正确的响应内容，并且与数据库管理系统协同保存前端的各类信息。云存储服务器是数据存储空间，所有上传的文件内容都保存在云存储空间。平台的结构框架如图 3-3 所示：

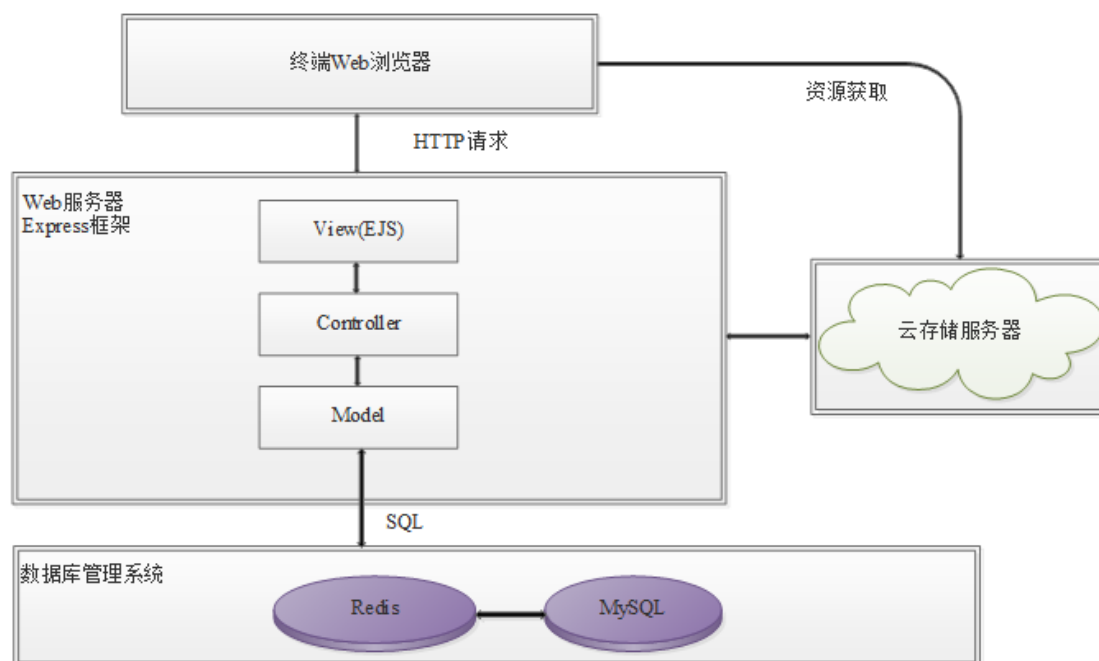


图 3-3 平台框架图

3.3 功能模块设计

结合需求分析，文件管理平台大致划分为四大模块：用户账号模块、文件管理模块、社交模块、社区互助模块。用户模块是包括用户注册、用户个人信息更新、用户密码找回；文件管理模块是平台的核心模块，包括文件夹管理，文件上传、下载、更新、删除以及在线预览等；社交模块是平台提供的在线交流、文件分享、消息管理的功能模块，社区模块允许使用者在社区内发布请求，分享资源等功能。平台的模块框图如 3-4 所示：

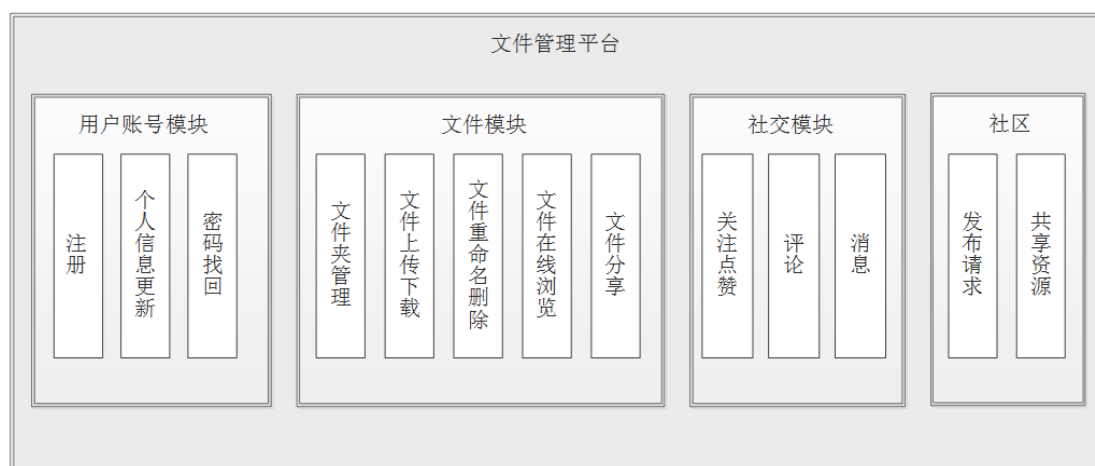


图 3-4 平台模块框图

3.3.1 用户账号模块

用户模块由注册登录、个人信息更新、密码找回三个功能组成。

1) 注册登录

用户需要使用平台上的功能，必须拥有进入平台的账号。用户可以进入注册页面，按照一定要求填写与账号相关的信息，例如注册邮箱、账号密码、性别等。填写完毕的注册信息，再通过表单发送给服务器，在提交之前，前端会检测注册信息的有效性以及唯一性，成功后，账号信息还会包括部分默认信息。在此过程中注册信息需要保证：

- (1) 唯一性：

注册邮箱是唯一识别，一个邮箱不能够重复注册，当出现这种情况，平台会跳转错误页面提示用户邮箱被重复注册。

(2) 有效性

前端需要检查注册信息填写的完整性以及格式的正确性，比如邮箱格式是否符合规范，用户信息填写是否遗漏等。如果信息填写不完整或信息错误，注册信息不会发送给 Web 服务器，相应的，前端会提示错误信息。

用户登录平台只需要填写注册的邮箱以及对应的密码，就可以进入平台使用其中的各类功能。用户登录成功后，Web 服务器利用 Redis 的存储服务，在会话期间实现 Session^[38]共享，解决 HTTP 请求过程的无状态难题。

2) 个人信息更新

注册过程中，部分信息并不强制要求填写，因此对于其他的个人信息要在登录平台后方可更新。详细的可更新信息见表 3-1：

表 3-1 个人信息更新表

名称	默认值	说明
用户名	-	该项为注册必填项
个人介绍	暂无个人介绍	
用户个人首页地址	-	常用的个人首页
性别	男	

3) 密码找回

用户在使用平台的过程中，可能忘记密码，进而无法使用平台。因此为了保证用户在丢失密码的情况下，仍能使用注册的邮箱账号进入系统，必须提供一定的容错功能，这就是找回密码功能。密码找回功能的实现需要前端、Web 服务器以及 SMTP(Simple Mail Transfer Protocol)^[39]相互协同完成。用户可以填写注册平台所使用的邮箱，Web 服务器通过 SMTP 发送一封找回密码邮件到注册邮箱^[40]，邮件内容中包含重置密码连接，连接中有唯一的重置码，利用这个连接可以定位到重置密码页，重新设置密码。重置成功后，用户即可以使用重置的密码和账号再次登录平台。设计流程如图 3-5 所示：

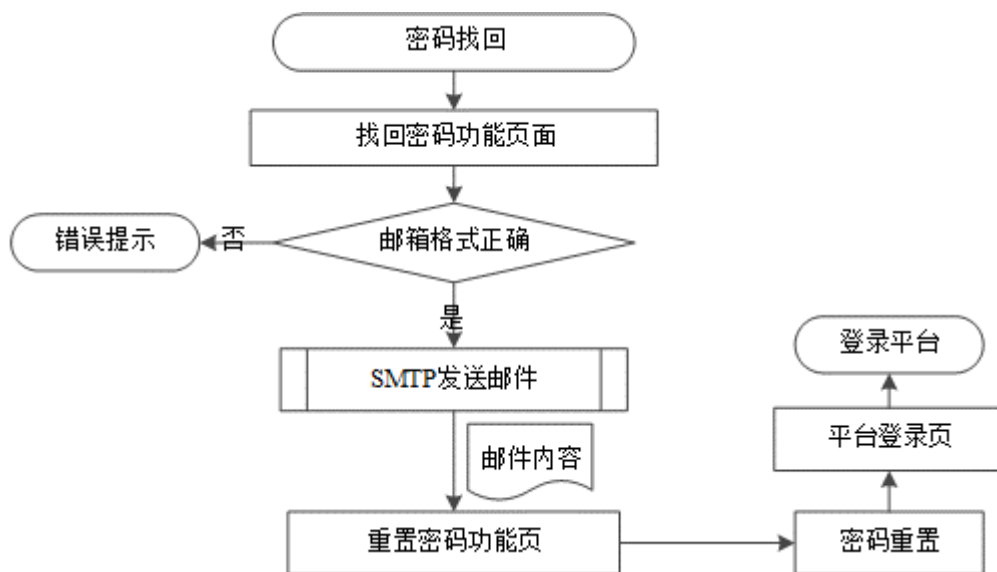


图 3-5 找回密码流程

3.3.2 文件模块

文件模块由文件上传下载、文件删除、文件重命名、文件在线浏览、文件分享、文件夹管理六个部分组成，模块功能划分如图 3-6 所示。

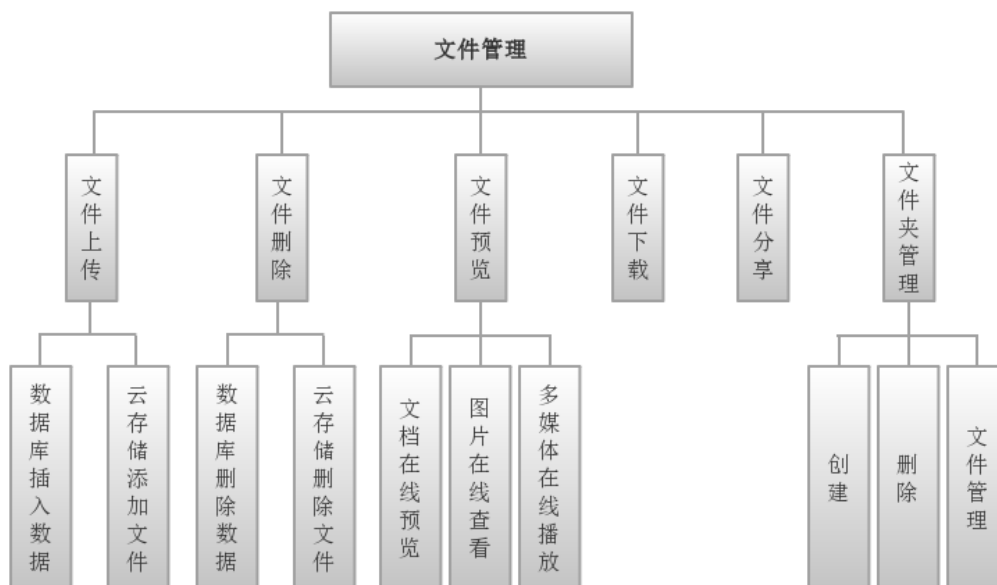


图 3-6 文件管理模块功能示意图

1) 文件上传

上传是平台的核心功能，平台主要目的就是利用云存储服务器将用户的各种类型的文件保存在线上，方便用户管理以及在线预览。根据不同的页面限制上传的文件类型，例如在内容管理页面可以上传任意类型格式的文件，包括文档类、图片、多媒体等；在文档页面上传的内容则被限制在文档范畴内，比如 Word、PDF、PPT 等；在音乐页面则只能上传音频格式文件，虽然 HTML5 在目前主流的浏览器中只支持 MP3、OGG、WAV，但是为了向后兼容，仍然可以上传 HTML5 播放器暂时无法播放的音乐格式；同样在视频文件显示页，只能上传视频文件。而且。上传功能支持多个文件以及多种格式同时上传，且优先使用 Ajax 方式上传，并在上传过程显示上传进度。成功上传的文件会保存到云存储服务器，而每个成功上传的文件信息会同步到后端的数据库。

2) 文件删除

文件删除功能允许用户将已经上传到文件管理平台上的文件删除，并且还需要支持多文件的删除。浏览器端使用 Ajax 发送一条删除请求到后端，服务器根据删除请求中的文件唯一标示键值，将存储在云存储的文件删除，然后根据云存储删除成功的回调信息将数据库中的对应数据清除，前端在删除成功后，将显示的文件条目移除。

3) 文件预览

使用文件在线预览功能，告别文件使用本地软件打开预览方式，同时减轻了不同文件类型使用不同软件预览的复杂度。文件预览要实现对不同格式文件的支持，包括音视频文件的在线播放，图片文件的预览、缩放以及多种文档格式内容的在线阅读。使用云存储服务的存储与数据处理功能，前端只需直接与云存储服务器交互，避免与 Web 服务器的交互，减轻业务服务器的负荷，进而更专注于业务逻辑的处理。

4) 文件下载

用户在需要使用平台上已经存储的文件或者其他用户分享的文件时，下载是实现这项功能的关键。利用 Web 前端技术的 iframe 以及 a 标签的 download 属性兼容各个浏览器下载功能，同时支持多文件的同时下载。

5) 文件分享

平台提供分享功能，用户上传的文件可以公开分享到平台上，让更多的用户可以使用分享的资源。分享的内容可以是文件、文件夹或者是两者的结合，用户可以对公开分享的内容进行在线预览、下载，还可以对其评论、点赞。用户也可以撤销分享，保证用户分享的自由性。

6) 文件夹管理

文件夹是文件与文件夹的集合，文件夹内存储的各类文件一方面可以使用上文提到的各类功能，比如文件的下载、预览、分享等。同时文件夹的管理还包含三个部分：文件夹的创建、文件夹的删除、文件夹内文件的管理。文件夹的创建使用 Ajax 向服务器发送一条 HTTP Post 请求，对应的页面会显示一条文件夹记录，文件夹可以点击编辑按钮更新文件夹的名称，文件夹的删除同样也是发送一条 HTTP Delete 请求，成功删除后，页面移除对应的文件夹条目。

3.3.3 社交互动模块

社交互动模块由点赞、用户关注、评论、消息通知四个部分组成，利用 WebSocket 的双向实时通信协议实现社交功能，一方面传送前端与后端交互的数据，另一方面实时分发产生的消息。下面进行详细分析。

1) 点赞

赞是平台用户对特定分享给予的评价，用户根据分享内容，如果喜欢分享的内容，或者分享的内容对自己非常有用，可以给予分享内容一个赞，作为给分享用户的奖赏，并且通过实时消息系统将赞消息推送给分享内容的用户。如果用户处于离线状态，在下次登录时，页面会显示数据库消息表中用户未查看的赞消息。用户也可以对已点赞的内容取消点赞，但是不会发送消息。

2) 用户关注

关注功能允许用户关注平台上的任何用户，进入用户的主页即可查看用户的个人信息并且可以关注用户，用户的主页记录了自己的关注用户，以及自己的粉丝集合。关注功能使用实时推送功能，关注的信息同步到关注数据表中，利用消息通知功能，实时地将关注消息推送给被关注用户。

3) 评论

针对平台上的分享内容，用户可以进行点评，所有的评论信息都会录入数据库评论信息表中，并且将评论信息实时推送给分享内容的用户，可以更好的与评论者进行实时讨论。

4) 消息通知

如上文所提到的，点赞、关注、评论三部分都是通过实时消息系统将消息分类保存到数据库中。如果用户在线，则直接将信息推送给接收用户，否则只能在下次登录平台时，查看接收到的所有信息，并且消息会通过类型的不同分类显示。

3.3.4 资源互助社区

资源社区是用户之间分享内容、发表见解的空间，用户在社区中可以发送请求帖子，请求需要的内容，也可以分享资源给其他需要的用户。每个请求帖子中包含了用户需要的文件资源的大致描述以及资源类型，每个用户都可以在社区中对帖子回复、讨论或者给予帮助。请求帮助用户可以实时查看自己的帖子是否有其他用户给予回应，也可以查看其他用户的帖子。

利用社区提供的功能，可以很好的帮助社区内用户之间的互助。社区的主要功能包括三部分：

1) 资源请求的发布

请求的发布是实现社区资源互助的起点，用户首先根据需求，描述希望解决的问题，然后通过平台发布到社区内。社区内每日浏览的用户会根据请求资源的描述给予帮助。

2) 资源请求的查看

用户可以在社区中浏览各类帖子，包括自己发布的以及平台上其他用户发布的请求帖。一方面用户可以在这些帖子中找到自己需要的资源，也可以提供资源帮助其他用户。

3) 资源的共享

资源共享是社区的目的，让每个用户都参与进来，最大限度帮助其他用户。共享方式有多种，包括分享存储在平台上的文件，也可以以链接的方式将站外的资源提供给需要的用户。

3.4 数据库设计

数据库作为存储、管理数据的仓库，主要实现信息的有效管理以及资源利用。在 Web 开发过程中，要实现动态页面显示，数据管理以及与 Web 前端的数据交互，数据库管理系统都是不可或缺的部分。

1) ER 图

根据系统中的整体功能逻辑，存储管理系统中主要包括的实体有：用户、上传文件信息、用户关系、内容分享、内容评论、消息、资源社区等。这些实体之间既独立又存在依存关系，一方面各自是独立的存储数据，另一方面共同合成数据集。其中的实体关系图如下 3-7 所示。

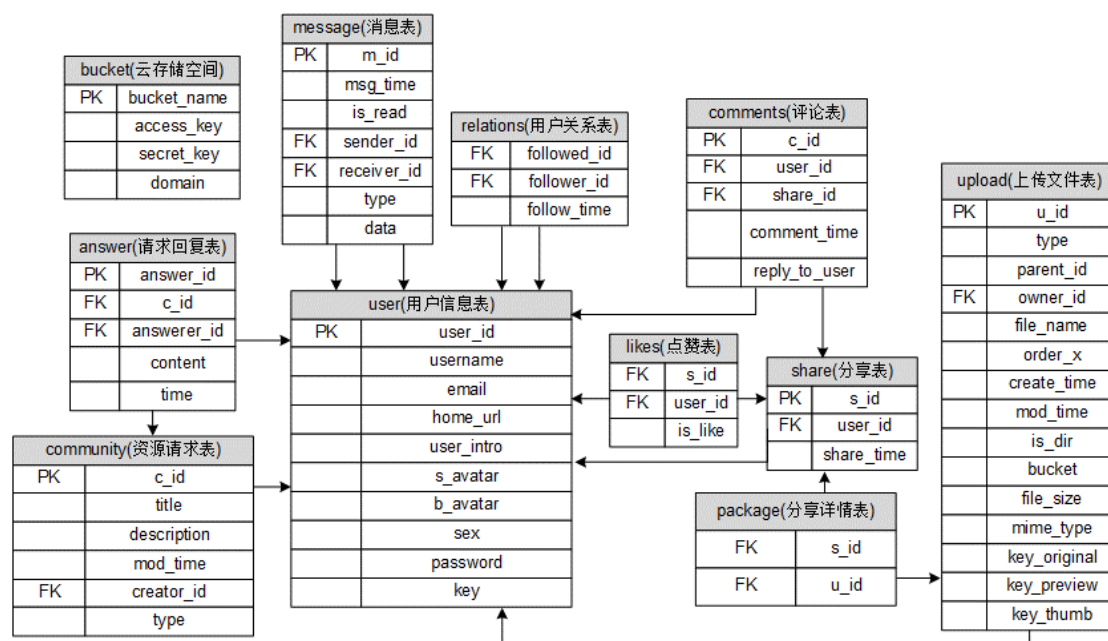


图 3-7 平台 E-R 图

2) 各数据表的设计

根据 E-R 图，文件管理平台中主要的数据表包括：用户表 user，上传文件信息表 upload，内容分享信息表 share，评论信息表 comments，用户关注表 relations，消息记录表 message，以及分享内容详情表 package，社区资源请求表 community，资源社区回复表 answers 等。数据表详细设计如下：

用户表 **user** 用户记录注册用户的相关信息,其中 **user_id** 是注册用户的唯一标识, **key** 值是存储在云储存服务器中头像文件的键值;利用这个 **key** 值获取用户头像、以及更新头像;登录密码是进入平台的凭证。部分字段详细描述如表 3-2:

表 3-2 用户表 **user**

字段名	中文名称	字段类型	备注
user_id	用户 ID	char(14)	PK
username	用户名	varchar(255)	
email	用户注册邮箱	varchar(255)	
s_avatar	小头像地址	varchar(255)	
password	登录密码	varchar(255)	
key	上传文件 key 值	varchar(255)	

文件信息表 **upload** 记录了文件的各种信息。所在文件夹 **ID** 指定上传文件所属的文件夹,默认为空。如果存在,表明上传的文件在此 **ID** 文件夹下。文件类型 **type** 指定文件所属类型,前端通过这个字段的值,渲染出不同的显示信息。**key_original** 是文件在云存储的标识,前端请求资源就是根据这个键值。其中的 **parent_id** 是文件所在文件夹的 **ID**,默认为 **null**。部分内容的详细介绍如表 3-3 所示:

表 3-3 上传文件信息表

字段名	中文名称	字段类型	备注
u_id	上传文件 ID	char(14)	PK
type	文件类型	int(11)	
parent_id	所在文件夹 ID	char(14)	
owner_id	上传用户 ID	char(14)	FK
file_name	文件名	varchar(500)	
mod_time	上一次修改时间	datetime	
is_dir	是否是文件夹	char(1)	
file_size	文件大小	bigint(20)	
mime_type	文件的 MIME 类型	varchar(255)	
key_original	原始上传 key 值	varchar(500)	

文件分享数据表 **share**, 记录用户分享内容的信息。数据表中包含三个主要字段, 分别是分享内容的 **id** 标示、分享用户的唯一识别字段 **user_id** 以及文件分享的具体时间。详细设计如表 3-4 所示:

表 3-4 内容分享信息表

字段名	中文名称	字段类型	备注
s_id	分享 ID	char(14)	PK
user_id	分享用户 ID	char(14)	
share_time	分享时间	datetime	

用户分享详细内容表 **package**, 记录 **share** 的内容详情, 一次分享中可能包含多个文件或文件夹, 所以 **package** 记录了分享中的详细内容描述, 记录了分享内容合集。详细的 **package** 数据表设计如表 3-5 所示:

表 3-5 分享内容详情表

字段名	中文名称	字段类型	备注
s_id	分享 ID	char(14)	FK
u_id	分享的文件 ID	char(14)	FK

评论信息表用于记录用户对某个分享的各种评论, 其中的评论 ID 表示一次特定的评论, **user_id** 指定了评论分享内容的用户, **share_id** 是评论的对象, 标示用户的某个分享, **comment** 是前端使用安全机制处理过的详细评论内容, 防止 XSS(Cross-Site Scripting 跨站脚本攻击)^[41]。详细的 **comments** 数据表设计如表 3-6 所示:

表 3-6 评论信息记录表

字段名	中文名称	字段类型	备注
c_id	评论 ID	char(14)	PK
user_id	评论者 ID	char(14)	FK
comment	评论内容	text	
share_id	分享 ID	char(14)	

消息记录表 **message**, 用于记录用户的各类消息, 消息表中的 **type** 标识不同的消息类型, 在消息的分发中使用这个字段区分不同的消息, **data** 记录了评论以及点赞内容的链接, 方便用户通过此字段直接找到分享内容, **is_read** 字段说明消息是否被

阅读, 默认情况下是 false, 表示消息没有被用户查看过, 而 sender_id 以及 receiver_id 分别表示消息的发送者以及接受者。相关 message 数据表设计如表 3-7 所示:

表 3-7 消息记录表

字段名	中文名称	字段类型	备注
m_id	消息 ID	char(14)	PK
is_read	消息是否已读	char(1)	
sender_id	发送者 ID	char(14)	FK
receiver_id	接受者 ID	char(14)	FK
type	消息类型	char(1)	
data	消息信息	text	

社区资源请求表记录着资源请求的详细描述。title 是请求的标题, 描述资源的大致概念, desc 是对请求内容更加详细的描述, type 表明此次资源请求的类型。更加详细的设计如表 3-8 所示:

表 3-8 资源请求表

字段名	中文名称	字段类型	备注
c_id	资源请求 ID	char(14)	PK
title	请求标题	varchar(255)	
desc	请求描述	text	
type	请求资源类型	char(14)	

社区资源请求回复表 answer, 用于记录每个资源请求的回复记录。answer_id 是用户某次特定回复的标示, 而 c_id 指定了一个社区内的资源请求, content 就是一条详细的回复内容。详细描述如表 3-9 所示:

表 3-9 请求回复表

字段名	中文名称	字段类型	备注
answer_id	请求回复 ID	char(14)	PK
c_id	资源请求 ID	char(14)	FK
answerer_id	回复用户 ID	char(14)	FK
content	回复内容	text	

3.5 本章小结

本章通过用户需求分析,确定了平台用户所需要的各项功能,包括文件的存储、逻辑处理、在线预览等。结合用户的需求,对平台所需功能进行了分析,确定文件管理平台的功能模块结构,进而确定了文件管理平台的整体框架。在整体框架的指导下,再分别详述了各个模块的设计逻辑、功能划分。最后由各功能需求的数据逻辑,确定了数据库的设计。详述了数据库的整体结构,以及各个数据表所对应的数据信息。

4 文件管理平台的实现与测试

上一章中, 根据需求分析对平台的整体功能、交互逻辑进行了分类, 设计出了平台各个模块结构以及各个模块的功能逻辑。平台的功能层次结构图如下 4-1 所示:

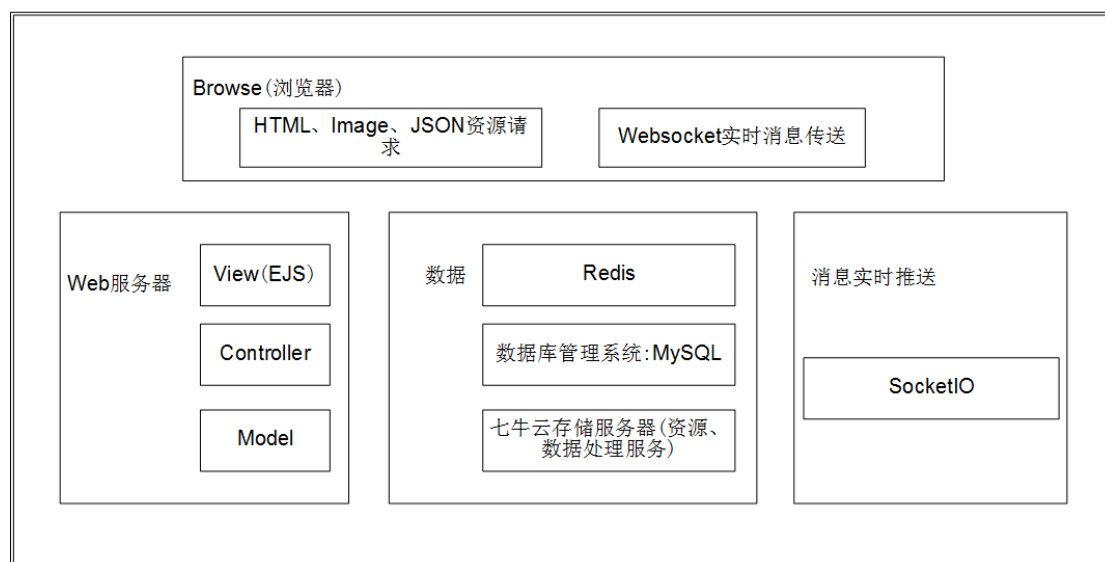


图 4-1 平台功能层次图

本章将从平台中挑选出重点的模块详述其实现, 最后从平台的功能上以及性能上做一个全面的测试。

4.1 文件管理模块

平台以文件管理为核心功能, 因此文件管理模块作为平台的核心功能模块包含了很多重要的组成部分, 包括文件夹管理、文件上传与删除、文件下载、文件在线预览以及内容分享。下面将详细介绍文件管理平台各功能的实现。

4.1.1 文件夹管理

文件夹是文件的集合, 可以合理的组织文件, 将某一类或者用户喜爱的文件收集到一个文件夹中, 方便用户管理。下面将分条实现文件夹的管理功能的各个部分。

1) 文件夹的创建

浏览器端通过 Ajax 发送一条创建请求, Web 服务器收到该条文件夹创建请求后, 在数据模型层利用 SQL 语句, 在上传文件数据表中添加一条创建记录。如果创建成功, Web 服务器返回给前端浏览器文件夹完整的创建信息, 前端通过 file_item.ejs 模板渲染显示新建的文件夹。如果不成功, Web 服务器会返回一条创建不成功的错误信息。其中完整的文件夹创建时序图如图 4-2 所示:

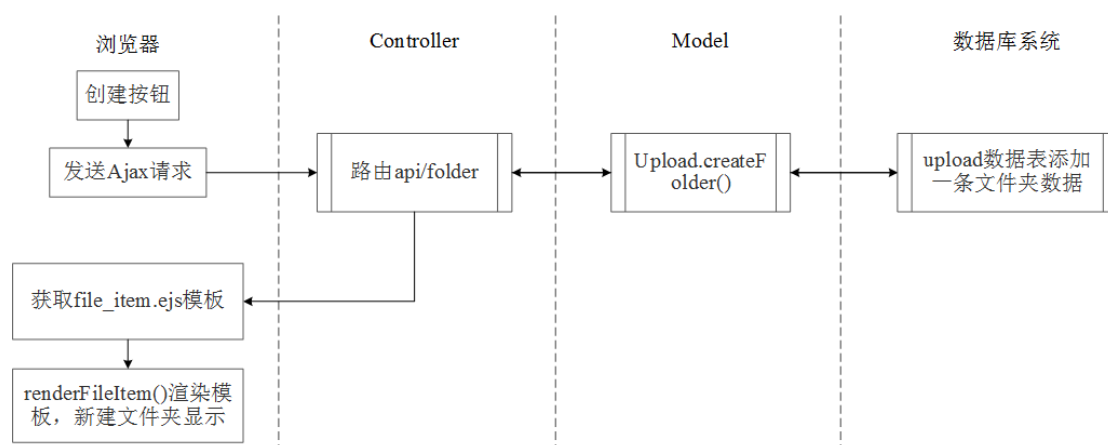


图 4-2 文件夹创建时序图

其中前端发送的 POST 请求经过路由定位, 被文件夹管理控制器截获, 控制器获取 POST 请求的文件夹数据, 调用 Upload 数据模型中的 createFolder 方法将文件夹信息写入 upload 数据表。写入成功后, 控制器会将数据表中完整的文件夹信息回馈给浏览器端, 浏览器接收到成功返回数据后, 通过 Ajax 请求获取 file_item 模板, 最后结合文件夹数据使用 renderFileItem 将渲染的内容显示。其中 Upload 数据模型层中文件夹创建的核心代码如下:

```

function createFolder(data){
    var folder = { type: 7, is_dir: 1 };
    _.extendOwn(folder, data); //合成文件夹数据
    insertSql = 'insert upload values(' + arrJoin(_.values(folder)).join() + ')';//创建文件夹 sql
    promise = new Promise(function(resolve, reject) {
        conn.query(insertSql);
    });
    return promise;
}
    
```

2) 文件夹的重命名

文件的重命名使用 Ajax，发送一条 put 请求，服务器控制层根据请求信息调用对应接口，更新 upload 数据表中的特定文件夹名称。其时序图如 4-3 所示：

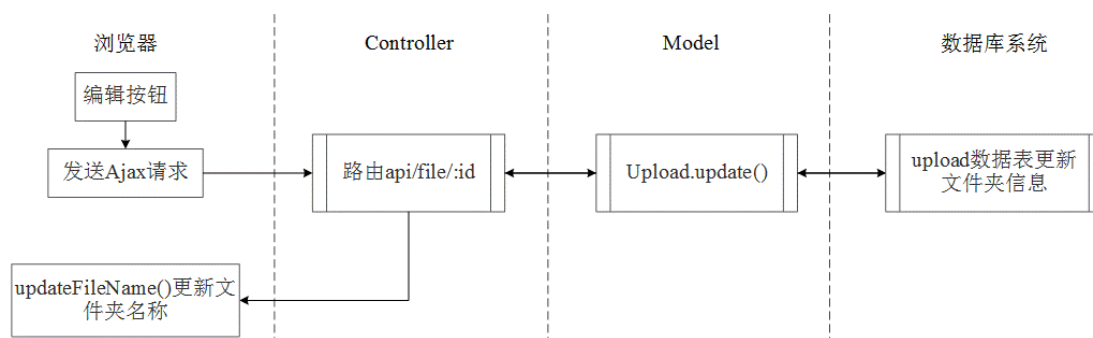


图 4-3 文件夹重命名时序图

前端的 PUT 请求携带文件夹的唯一标示 id 以及重新定义的文件名称，Web 服务器接受到请求后，使用 Upload 模型的 Upload.update 接口更新指定的文件夹名称。

3) 文件夹的删除

文件夹的删除不仅仅只是删除特定的文件夹，还需要根据文件夹是否含有其他内容，做不同的处理。如果文件夹中没有其他的文件数据，则只是简单的删除文件夹，但是文件夹中可能包含其他文件，因此在删除文件夹的同时还需要删除文件夹内的其他内容，这是删除中的一个难点。其中删除过程的时序图如 4-4 所示：

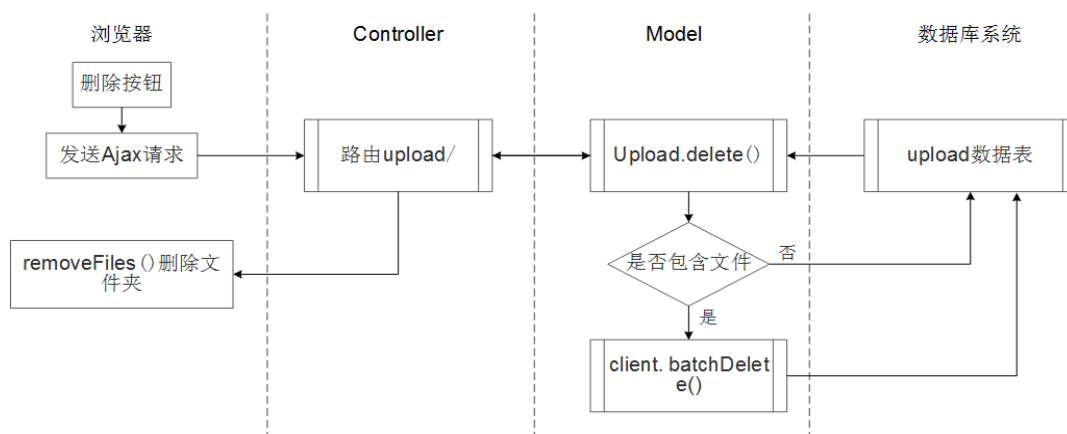


图 4-4 文件夹删除时序图

浏览器利用 Ajax 将选中的文件夹信息发送给服务器，Web 服务器接受到请求后，判断删除的文件夹是否包含其他文件或者文件夹，如果没有包含其他内容，直接调用文件夹删除接口从数据表中删除文件夹条目，如果包含其他文件，则需要将调用云存储的 API，删除保存在云存储上的文件，如果包含文件夹，则使用递归的方式删

除文件夹及其内部包含的文件内容。删除成功后，前端调用 `removeFile` 方法移除页面中的文件夹。

4) 文件夹内文件的管理

文件夹内的文件管理同样包括文件上传、删除、文件重命名，与下文所述的内容基本一致，因此此处不再赘述。

4.1.2 文件资源管理

1) 文件上传

利用七牛提供的 `QiniuJsSDK` 软件开发包，优先使用 `HTML5` 提供的 `XMLHttpRequest2.0` 接口实现异步文件上传。如果浏览器不支持，再降级用 `flash` 实现本地文件到云存储服务器的上传。上传流程如图 4-5 所示：

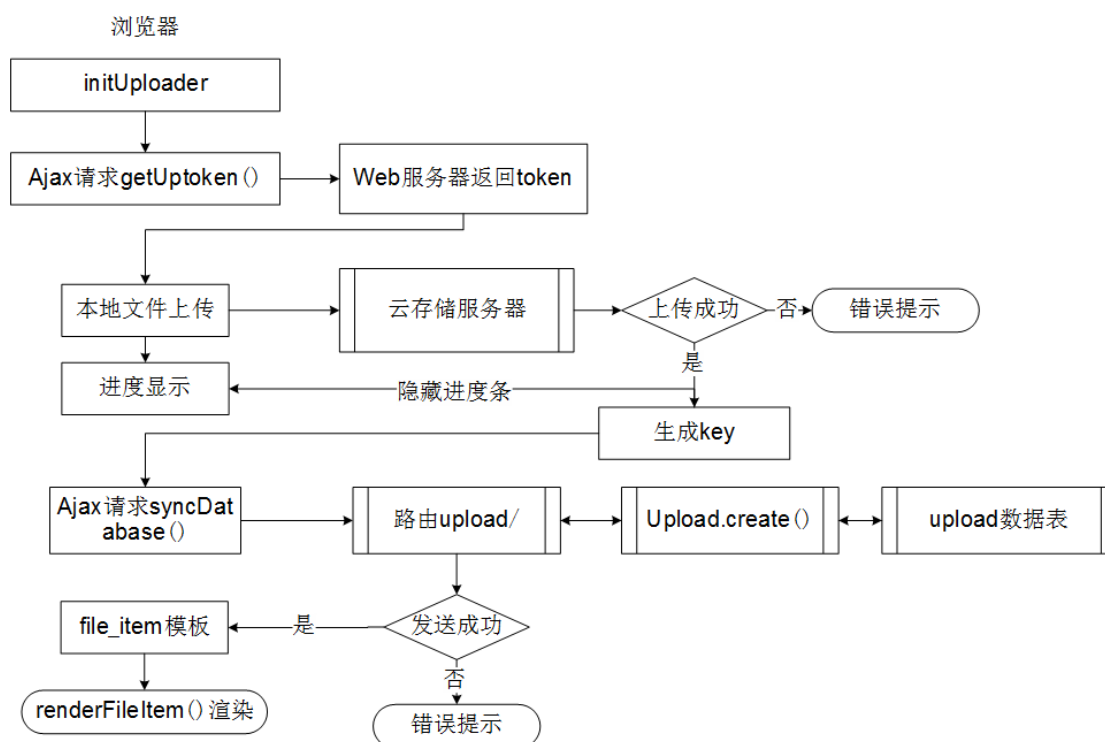


图 4-5 文件上传流程图

首先，`QiniuJsSDK` 在浏览器端调用 `initUploader` 初始化上传功能，利用 `Ajax` 从 `Web` 服务器获取到上传凭证 `uptoken`。利用这个 `uptoken`，前端就拥有了将本地的文件上传至云存储服务器中权限，上传文件完成后，云存储服务器会生成指定文件的

唯一标识键值，利用这个键值浏览器就可以访问云存储的文件资源。浏览器再将文件信息以及生成的 key 值再利用 Ajax 请求方法 syncDatabase 将文件数据同步到数据表中。最后，Web 前端利用 file_item.ejs 模板渲染上传文件条目。Web 前端文件上传的核心代码如下：

```
function initUploader(opts) {  
    var defaultOpts = { //默认上传配置参数  
        max_file_size: '100mb',  
        chunk_size: '4mb',  
        uptoken_url: '/upload/uptoken',  
        ...  
    };  
    $.extend(opts, defaultOpts); //合并上传配置  
    return Qiniu.uploader(opts); //执行上传操作  
}
```

上传文件大小需要低于 100MB，如果超过会给出提示信息。文件上传过程支持分片上传机制，如果上传文件大于 4MB，则文件以每 4MB 大小分片上传。QiniuJsSDK 提供了可配置参数，可以在其他的需求下，使用 \$.extend(opts, defaultOpts) 修改默认配置，opts 中的配置会覆盖默认设置。

其中文件上传后端数据同步的核心代码如下：

```
var files = req.body['files[]'];  
files = files.map(function(doc) {  
    file = JSON.parse(doc); //解析文件信息  
    return $.extend(doc, options);  
});  
Upload.create(files).then(function(contents) { //同步上传文件信息  
    res.json(contents);  
});
```

上传过程中，支持多文件上传，这样浏览器端传递给 Web 服务器的同步数据也会出现多条。因此在服务器端，首先获取文件信息数组 files，再使用 Upload.create 同步到上传数据表 upload 中，并以 JSON 数组形式回传文件内容 contents 给浏览器端。

2) 文件删除

上文文件夹管理模块功能中，从时序图和功能实现上详细描述了文件夹删除的过程。文件的删除与文件夹删除共用同一个接口，文件夹的删除作为文件删除的特殊部分，包含了文件删除。只是文件的删除不需要使用文件夹删除中的文件夹递归功能，直接将选中的文件的键值和 ID 属性通过 Ajax 请求发送给 Web 服务器，Web 服务器批量删除存储在云服务器上的资源，并更新数据库的上传数据表。因此这里不再赘述文件删除功能。

3) 文件下载

平台上用户上传的文件在需要时可以下载到本地，其他用户分享的内容也可以直接下载到本地，方便用户使用。文件的下载利用前端技术直接从云存储服务器下载。为了消除多浏览器之间的差异，前端需要适配各类浏览器，使用浏览器中的 userAgent 属性判断浏览器类型，进而采用适用不同浏览器的下载方法。其下载的核心代码如下：

```
var ua = navigator.userAgent.toLowerCase(), //根据不同浏览器采用不同下载方式
    IEDown = /trident|msie/.test(ua),
    FFDown = /firefox\\([\\d.]+)\\.test(ua);
downer = IEDown ? IEDownloadFile : FFDown ? fireFoxDownload : downloadFile;
```

4.1.3 内容分享

文件的分享功能指的是用户将自己的文件分享到公开空间，所有平台上的用户都可以查看分享的文件内容，进行线上预览、下载以及对其评论等。其功能时序图如下 4-6 所示：

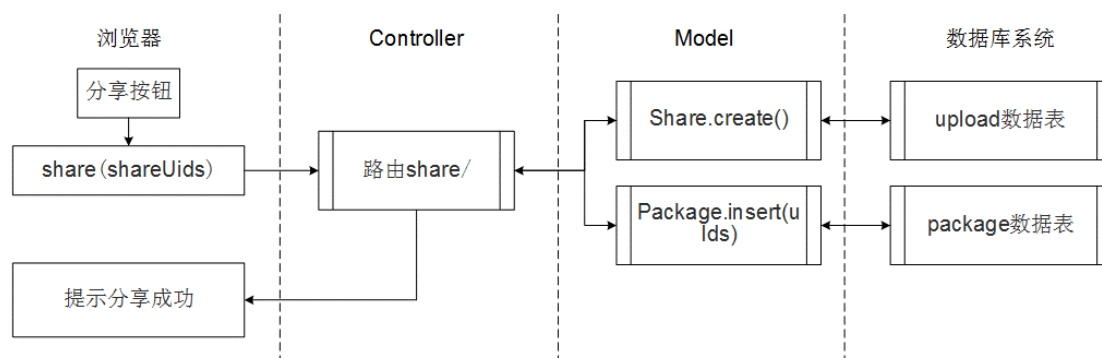


图 4-6 文件分享时序图

浏览器点击分享按钮后，前端逻辑收集所有被选中文件以及文件夹，使用 `getFileUids` 获取要选择分享的文件或者文件夹 ID。然后使用 Ajax 请求 `share(shareUids)` 发送一条创建分享 HTTP 请求。在控制层监听浏览器端的 Post 请求，然后将分享的文件 ID 列表交给后端逻辑。数据模型层 `Share.create()` 创建一条分享记录，然后将分享的文件信息使用 `Package.insert(uids)` 同步到分享详情表 `package` 中。其中数据同步代码如下：

```
connection.beginTransaction(function(){
    Share.create(); //同步数据库文件分享信息
    Package.insert(uids);
});
```

文件夹的管理、文件的上传、下载、删除与文件分享功能是文件管理平台的基础部分，其页面实现图如下 4-7 所示：



图 4-7 文件管理页面实现图

4.1.4 多类型文件在线预览

平台上的文件根据不同格式分成五类：图片类型、文档类型、音乐类型、视频类型以及其他文件类型。对于不同的文件类型，前端使用 HTML5 技术进行浏览。

1) 图片预览

图片类型的文件预览内容包括缩略图以及原始图片，点击缩略图，对应的原始图显示在正文显示区，原始图还提供缩放拖动功能，原始图的缩放使用按比例缩放，缩放的区间在 25%~200%。原始图片显示在显示区的正中间，当图片内容超过显示区时，用户可以拖动图片到指定部分。其中预览效果如图 4-8 所示：

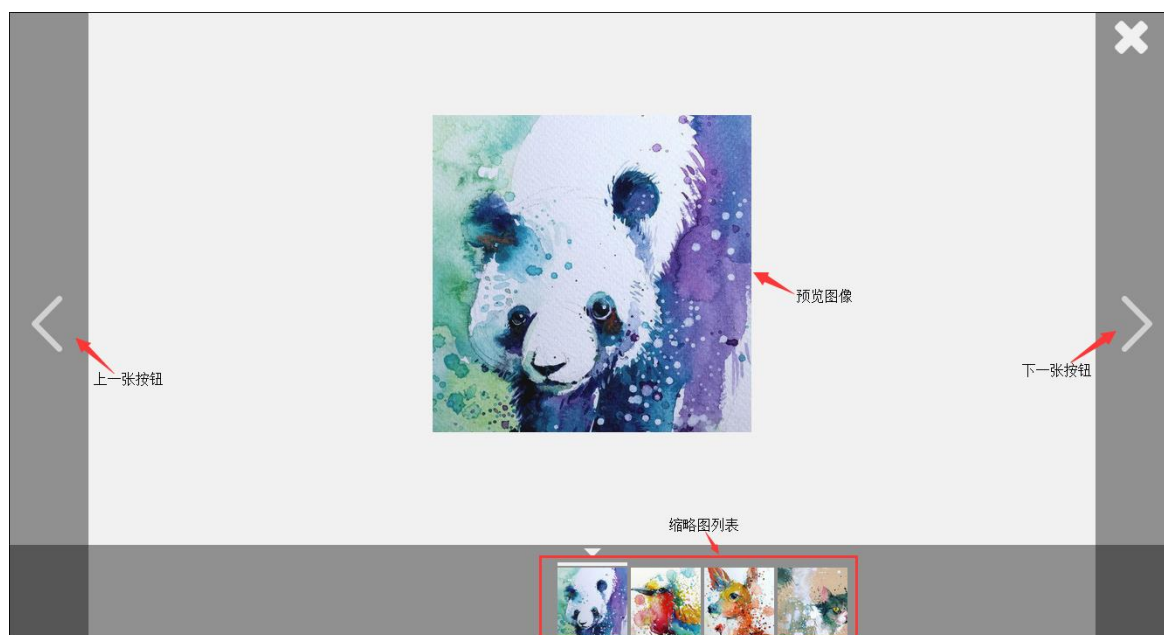


图 4-8 图片预览页面图

2) 文档预览

文档类型中又包含各种不同类型的文件类型，七牛云存储服务提供的强大的文件处理功能，可以将文档类型的文件转化为 jpg 图片格式，前端获取转换后的图片文件，再在前端实现的预览器中进行预览。下面将利用 PPT 文件的预览来讲解文档预览的实现。首先前端从 Web 服务器获取文件的基本信息，包括文件名、存储在云存储的唯一标识键值，利用这个键值以及云存储的数据处理功能，获取处理后的 URL，按需加载资源。在 PPT 预览中，预先加载 PPT 文件的前 5 页，如果继续浏览，则每次都加载 5 页，直至 PPT 文件加载完成，预览区监听滚轮的滚动，进行上一页与下一页显示。PPT 文件的缩略图利用延迟加载，初始的缩略图使用 base64 数据格式，显示一张 150*115 的图片。当滚动到对应的缩略图位置后，浏览器才向云存储服务请求真实的缩略图片。其中 PPT 预览页面实现如图 4-9 所示



图 4-9 PPT 预览页面

3) 多媒体预览

利用 HTML5 的音视频操作接口,在浏览器端使用纯前端技术实现多媒体的播放成为可能。本段主要通过音频播放的实现来介绍多媒体播放功能。音频播放器使用 HTML5 提供的编程接口实现,播放器从云存储服务获取指定的音频文件,点击播放后,如果音频文件已经加载,则播放文件,否则等到音频文件已经加载后才可以播放。如果播放过程中出现错误则跳转到下一首。音频播放器提供音量控制、前一首、下一首、暂停、播放、播放进度控制等功能,使用这些控制功能可以播放 HTML5 播放器支持各类音频文件,并且控制音频的播放。如图 4-10 为个人音频页面的音频播放效果图:



图 4-10 音频播放功能实现页面

4.2 基于 WebSocket 的社交互动模块

社交互动模块是平台用户之间交流的一种渠道，平台的用户可以关注其他用户，点评其他用户的分享内容，对于自己喜欢的内容还可以点赞。社交过程中产生的信息会通过 WebSocket 进行实时的消息通知。

社交互动模块的主要功能包括：关注、点赞、评论以及消息分发。其逻辑结构如图 4-11 所示：

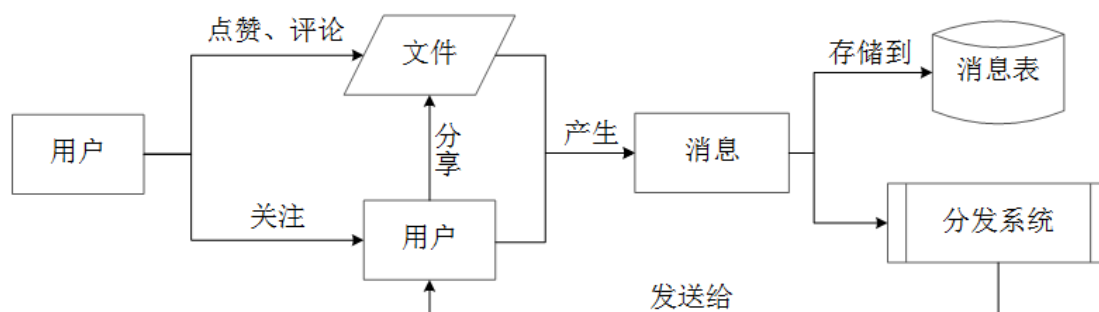


图 4-11 社交模块逻辑结构图

4.2.1 用户关注

平台上的用户可以关注其他用户，在用户主页可以查看到关注者列表，以及被关注者列表。浏览器端使用事件监听机制，监听关注按钮点击事件。一旦点击事件触发，前端调用 `getFollowData` 方法获取关注的信息，包括发送者 ID 信息 `sender_id`，接收者的 ID 信息 `receiver_id`。数据准备完毕后，利用创建的双向连接，发送一条 `follow` 消息，Web 服务器利用 `socket.on(follow)` 监听 `follow` 消息，一旦有消息到来，首先将关注信息存储到关注信息表，然后检测接收者是否在线，如果在线则发送一条 `msg` 类型的信息，告知接受者有一条消息，接收者在浏览器端获取信息后利用 `updateStatus` 更新消息显示记录，发送者刷新页面的关注显示。如果接收者不在线上，只更新发送者的页面显示。关注过程如图 4-12 所示：

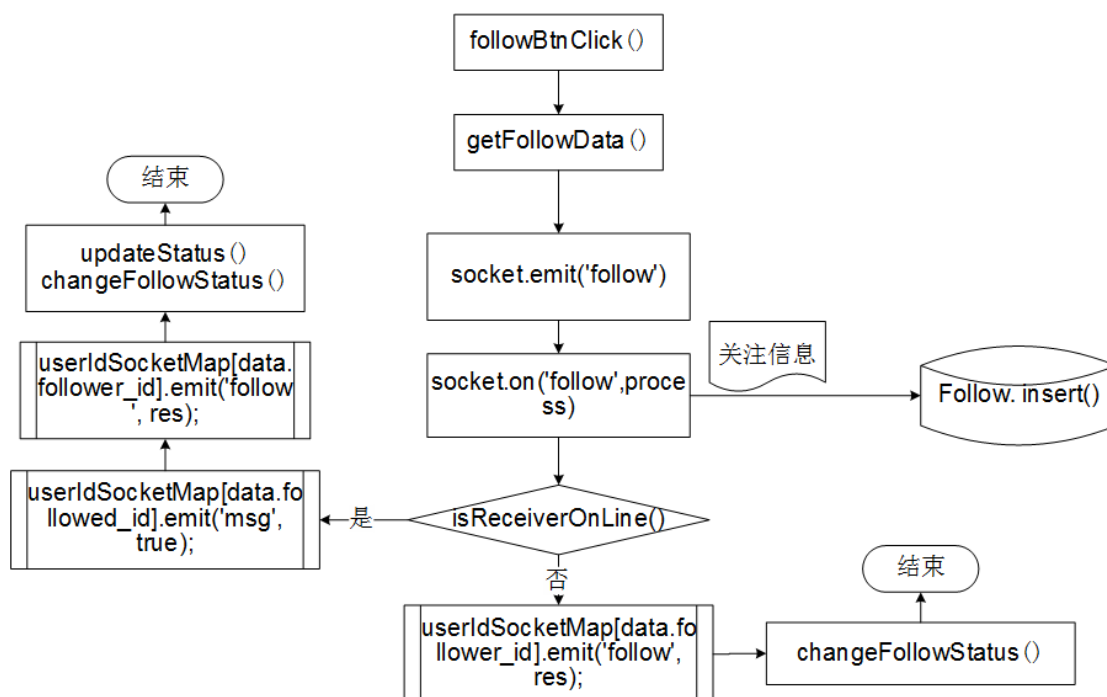


图 4-12 关注与取消关注流程图

4.2.2 分享内容点赞

平台上的分享内容用户可以在线预览、下载并且给予肯定。点赞就是对用户分享内容的肯定，表示用户对分享内容的喜爱。点赞消息通过建立的 socket 传送给 Web 服务器后，服务器会将点赞信息写入 like 表中，同时 isReceiverOnline 函数检测分享内容的拥有者是否在线，如果在线，则将消息实时推送给接收者，并给发送者回传一条成功信息，更新前端内容。否则只回传给发送者一条发送成功信息，更新前端显示。其中点赞流程如图 4-13 所示：

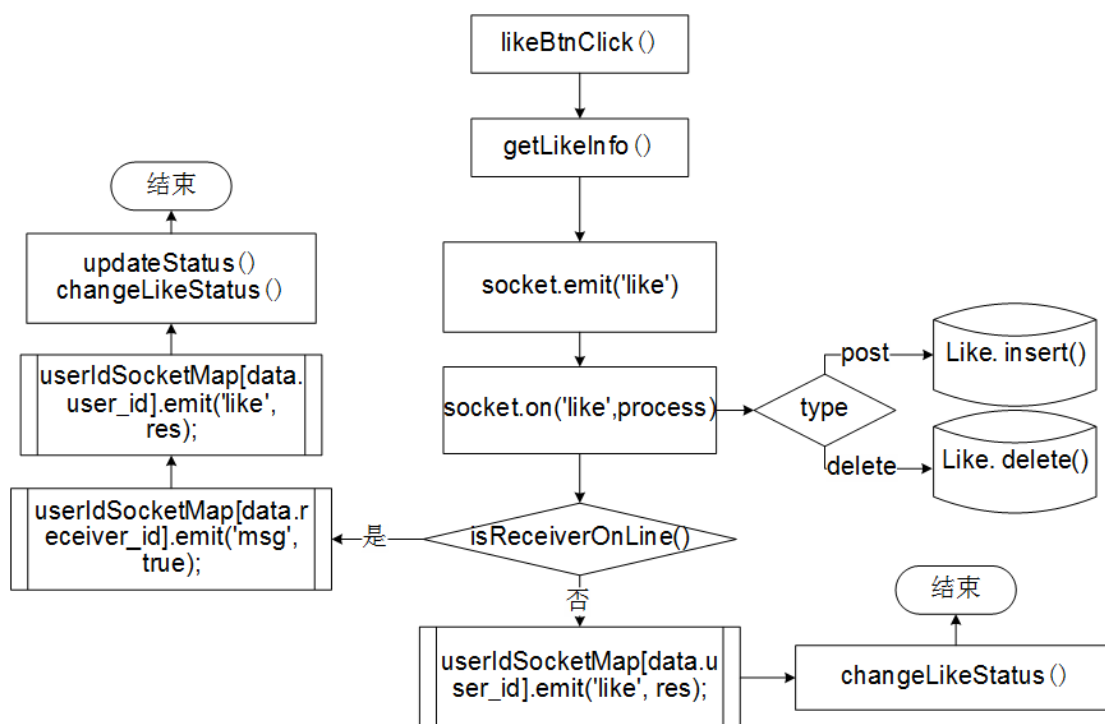


图 4-13 点赞功能流程图

4.2.3 分享内容评论

用户可以评论分享内容，评论的消息会通过实时系统发送给 Web 服务器，服务器接收后，将数据存储到评论消息表中，并分发给接收者。评论内容发送给服务器后，Web 服务器会将评论内容写入 comments 数据表中，同时 isReceiverOnline 函数检查分享内容的用户是否在线，如果在线，userIdSocketMap 中就会保留有在想用户的双向连接，通过这个 socket 连接给接收用户发送一条评论消息，并给发送者回馈一条评论成功信息，客户端更新评论内容。其中具体的评论实现流程图如下 4-14 所示：

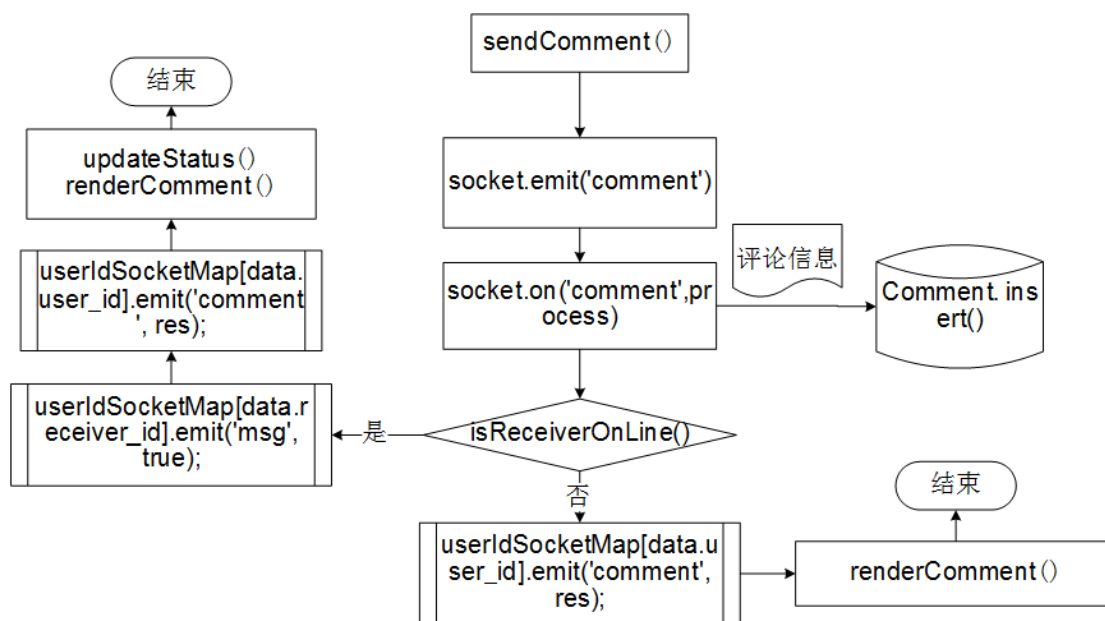


图 4-14 评论流程图

4.2.4 实时消息分发

上文中提到的点赞、关注、评论社交功能都会产生对应的信息，服务器根据消息的类型不同，将消息实时推送给接收者，消息的分发利用 NodeJS 平台建立一个消息服务器，消息内容存储到数据库，而消息在用户与消息服务器之间传送。消息分发网络图如下 4-15 所示：

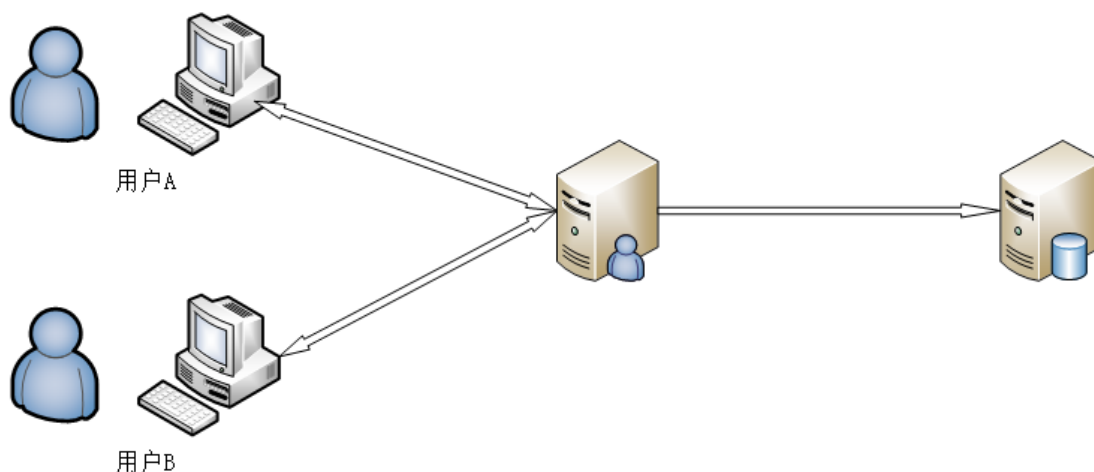


图 4-15 消息分发网络图

服务器端使用 Socket.IO 初始化一个消息服务模块，模块根据消息类型不同，分别使用不同的处理消息方式，并对消息进行存储与分发。实现的核心代码如下所示：

```
io.on('connection', function(soc) {  
  soc.on('socket', function(data) { //监听双向连接的建立  
    socketIdUserMap[soc.id] = data.user_id;  
    userIdSocketMap[data.user_id] = soc; //存储连接对象备用  
  });  
  initLike(soc); initFollow(soc); initComment(soc);  
  soc.on('disconnect', function() { //监听连接断开事件  
    delete userIdSocketMap[socketIdUserMap[soc.id]];  
    delete socketIdUserMap[soc.id];  
  });  
});
```

其中 `initLike` 方法处理与分发赞消息，`initComment` 方法处理评论消息，`initFollow` 方法处理用户关注消息。

前端也会利用 Socket.IO 框架构建的实时消息功能处理模块，与服务器端进行全双工通信。Web 前端消息处理功能的核心代码如下：

```
function initWebSocket() {  
  socket = SocketIO(Const.s_message); //建立 WebSocket 连接  
  socket.on('comment', listenComment.bind(socket)); //监听各类消息  
  socket.on('like', listenFollow.bind(socket));  
  .....  
  return socket;  
}
```

代码中，各个消息监听函数处理不同类型的消息，例如 `listenComments` 处理评论消息，`listenFollow` 处理用户的关注消息。消息显示实现页面如图 4-16 所示：

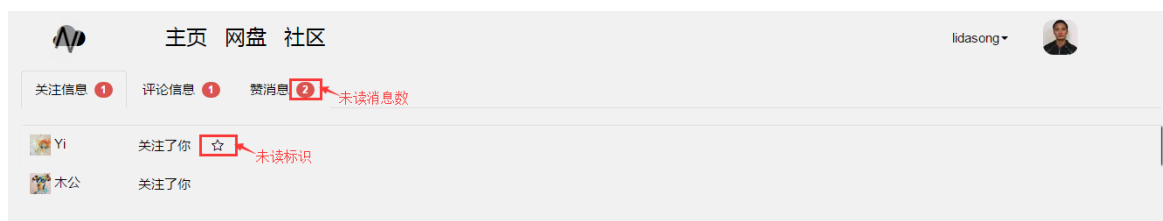


图 4-16 消息界面实现图

4.3 基于社区的资源互助

社区资源互助功能由三部分组成：发布资源请求、浏览各类请求帖子、共享资源。其中的社区功能结构如下 4-17 所示：

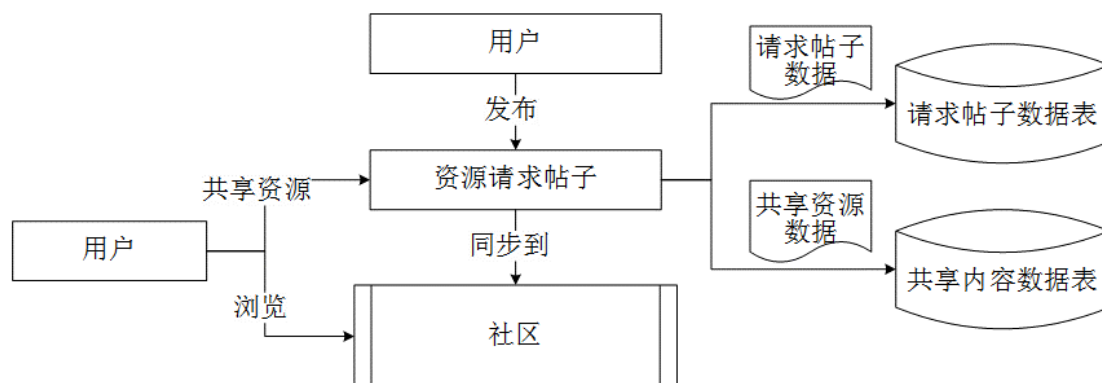


图 4-17 社区功能结构图

1) 资源请求发布

资源的请求信息通过 Ajax 发送给后端，服务器解析后，将数据同步到 community 数据表中，成功后页面跳转到社区页面，可以查看已经发布的资源请求。其中服务器端的核心代码如下：

```
router.post(function(req, res) {
    demand = xss(req.body); //编码请求内容，防止 XSS 攻击
    Community.create(demand).then(function(results) { //创建一条请求帖
        res.redirect('/community');
    });
});
```

其中的请求资源数据 demand 中包括请求资源的标题、描述、类型以及请求用户的 ID，通过 XSS 模块对 demand 数据处理，防止跨站脚本攻击^[42]。然后再将发布的请求同步到 community 数据表中。发布成功后 res.redirect('/community')跳转到社区页面。其中发布请求页面实现图如 4-18 所示：



图 4-18 请求发布页面实现图

2) 浏览社区请求帖子

用户发布的所有请求帖子都会同步到社区，利于其他用户查看或者进行资源互助。其中页面显示的核心代码如下：

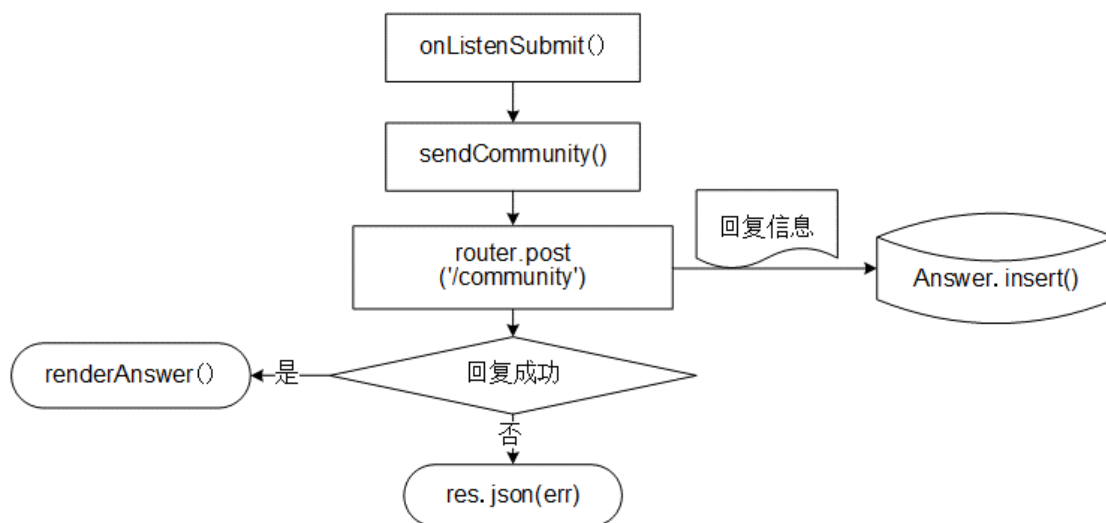
```
router.get('/community', function(req, res) {
  Promise.all([Community.get(),
    Answer.getAnswerCounts(); //获取请求帖中的数据
  ]).then(function(results) {
    res.render('community', results); //将请求页面内容发送给浏览器
  });
});
```

将请求帖子的请求详情以及其他用户给予的回复数显示在社区页面中，Community.get 从数据表 community 中获取一定数量的请求帖子，每个请求帖子包括创建时间、回复数、创建者等数据。Answer.getAnswerCounts 从数据表 answer 中找出对应请求帖的回复数，然后将结果页面回送给浏览器。

3) 资源共享

社区空间罗列出用户发布的各类请求帖，所有的平台用户都可以浏览到。因此当其他用户发现自己有相关资源时，就可以提供给请求用户，更好地帮助请求用户解决困难。资源的共享不仅仅可以让用户提供自己在平台上的资源，还可以将站外的有关资源提供给请求用户，通过一条链接导航到站外的具体内容，实现跨平台资源共享。

前端回复支持图片、连接以及其他 HTML 内容，后端数据库存储编码后的回复内容，最后前端显示提交的内容。具体流程如下 4-19 所示：



4-19 资源共享流程图

前端 onListenSubmit 监听提交按钮的点击事件，首先获取表单提交的分享内容 data，然后通过 sendCommunity 将内容上传到 Web 服务器，并使用 Answer.insert 将内容存储到 answer 数据表中。提交成功后，renderAnswer 渲染回复内容，否则服务器端响应错误消息。页面实现图如 4-20 所示：



4-20 资源共享页面图

4.4 平台测试

在平台的功能模块完成后，平台可用性、稳定性还需要经过严格的测试，只有在功能和性能达到了一定指标，才能保障各项功能可以长期平稳的运行。

4.4.1 测试环境

平台部署到阿里云服务器 ECS(Elastic Compute Service)，七牛云存储提供文件管理平台的文件存储与资源处理功能。部署结构图如下 4-21 所示：

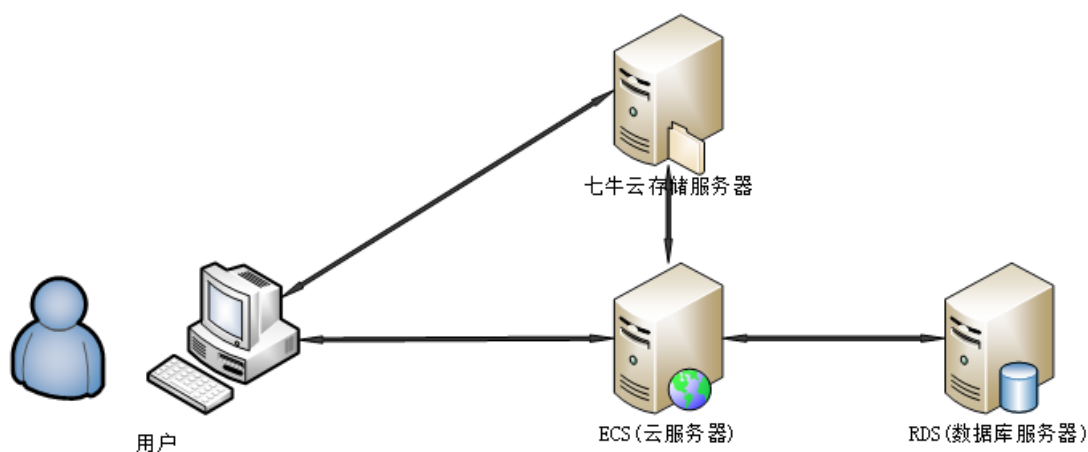


图 4-21 平台部署结构图

其中部署在线上的各项参数见表 4-1：

表 4-1 测试配置参数表

主机类型	软/硬件配置	参数
服务器	CPU&RAM	单核 RAM 1GB
	操作系统	Linux CentOS 7.0 64 位
	Web 服务器	NodeJS 4.2.2
	数据库服务器	MySQL 5.6.28
	域名	nodeyun.site
客户端	操作系统	Microsoft Windows 7 Service Pack 1
	浏览器	Chrome 、 Mozilla FireFox
		Internet Explorer 11、搜狗浏览器

4.4.2 功能测试

根据不同的功能模块，首先测试用户模块。主要测试内容包括注册、登录、忘记密码、编辑个人信息等。测试结果如表 4-2 所示。

表 4-2 用户模块测试表

序号	测试内容	设计结果	实际结果
1	用户注册页面	显示包括邮箱、用户名、密码的注册表单	正常
2	点击注册按钮	进入登录页面	正常
3	点击登录	进入首页，显示前 10 条内容	正常
4	忘记密码	显示注册邮箱输入框、发送邮件按钮	正常
5	点击发送邮件	提示邮件已发送	正常
6	点击重置密码按钮	成功设置后跳转到登录页	正常
7	点击用户信息修改按钮	显示目前用户信息表单	正常
8	修改个人信息，并点击保存	表单隐藏，页面信息更新	正常

然后测试文件管理模块，根据模块的组成，主要测试文件的上传、文件的下载、文件的重命名、文件的删除、文件预览以及文件分享内容。结果如表 4-3 所示：

表 4-3 文件管理模块测试表

序号	测试内容	设计结果	实际结果
1	点击上传按钮	显示上传对话框	正常
2	点击添加文件按钮	弹出文件资源管理器，选择文件，显示进度条	正常
3	成功上传	页面渲染文件信息	正常
4	点击编辑按钮	文件条目隐藏，对应位置显示输入框	正常
5	键入 Enter 键	文件条目显示，并显示更新的文件名	正常
6	点击删除按钮	选中文件条目从页面删除	正常
7	图片文件预览	居中显示图片，可以缩放、拖动	正常
8	音频文件预览	显示播放器，播放点击的音频文件	正常
9	文档内容预览	跳转到不同的文档预览页	正常

其次是文件夹的管理，文件夹测试内容包括，文件夹新建、文件夹的重命名、文件夹的删除、文件夹内上传文件，测试结果如表 4-4 所示：

表 4-4 文件夹管理测试表

序号	测试内容	设计结果	实际结果
1	点击新建文件夹	页面显示新建的文件夹，默认名为新文件夹	正常
2	点击删除文件夹	文件夹及其包含的内容都被删除	正常
3	点击编辑按钮	文件夹隐藏，对应位置显示输入框	正常
4	输入框键入 Enter	文件夹显示，并显示更新后的文件名	正常
5	点击上传按钮	文件上传，显示成功上传的文件信息	正常

最后是社交模块，社交模块的测试需要至少需要两个用户，测试内容包括点赞、分享评论、用户关注。如表 4-5 所示：

表 4-5 社交模块测试表

序号	测试内容	设计结果	实际结果
1	点赞	更新按钮内容，接收者显示消息提示	正常
2	点击关注按钮	更新关注信息，接收者显示消息提示	正常
3	点击评论按钮	分享内容下方显示评论输入框	正常
4	点击确定	评论内容显示在评论区，接收者接收提示消息	正常

4.4.3 性能测试

(1) 单页面加载性能测试

WebPageTest^[43]是一款功能强大的 Web 性能测试工具，而且已经开源。WebPageTest 提供了页面加载需要的详细数据，包括加载时间、各个文件的瀑布流图、每个资源获取的时间线等。WebPageTest 还提供了友好的测试结果图表，包括加载资源生成的瀑布图，通过瀑布图可以查看页面的每一个资源的时序图^[44]。下面将利用 WebPageTest 测试平台首页(<http://nodeyun.site/home/all>)的各个性能指标。包括连接建立时序图，以及网站资源文件的瀑布流图等。详细测试结果如下图 4-22 所示：

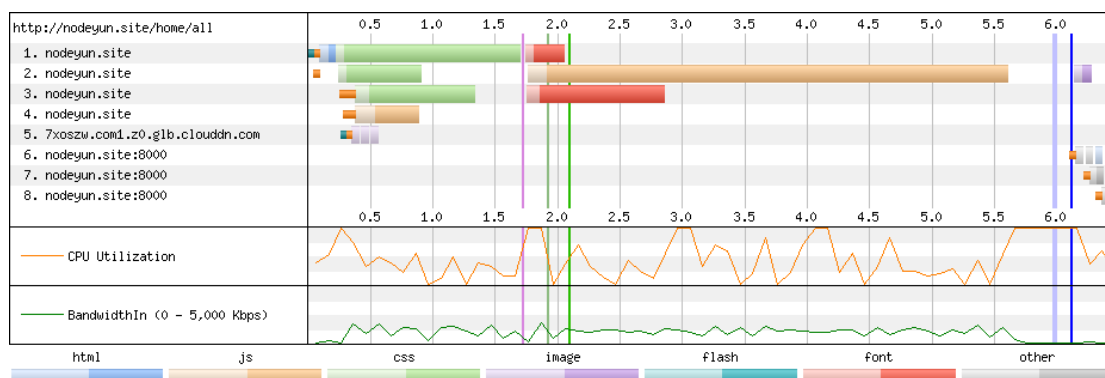


图 4-22 连接建立时序图

连接建立时序图表示下载整个页面过程中的发出的所有请求序列。上图中表示在请求 `http://nodeyun.site/home/all` 页面过程中向 `nodeyun.site` 发出四条请求，向 `nodeyun.site:8000` 发送 3 条请求，向七牛发送一条请求。

瀑布流图表示在页面请求过程中所有数据加载时序图，图 4-23 显示的有 CSS、JavaScript 等资源加载的过程图。

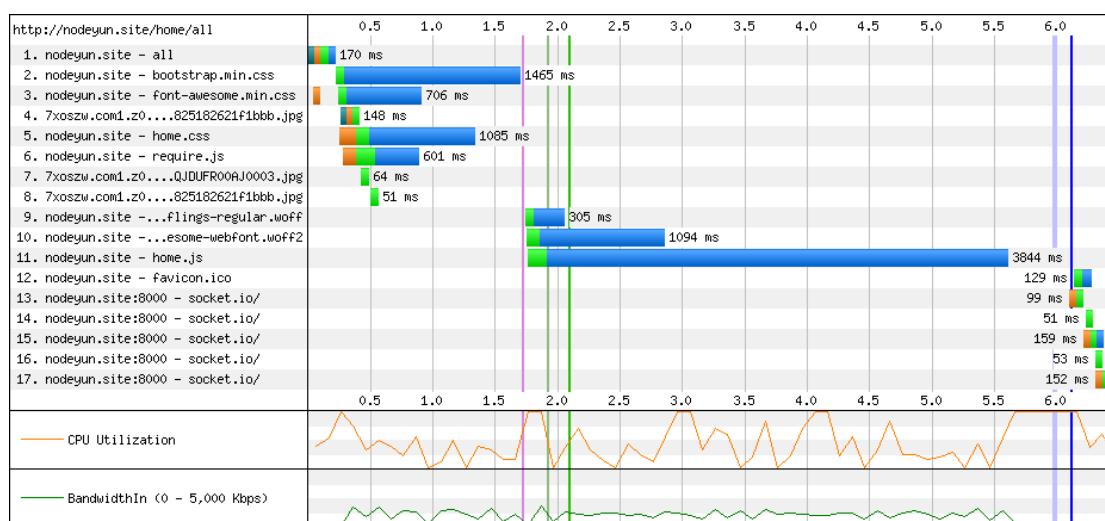


图 4-23 首页测试结果瀑布流图

下图 4-24 是 WebPageTest 在测试过程中获取的所有详细数据，包括第一次加载与重复加载的对比数据。

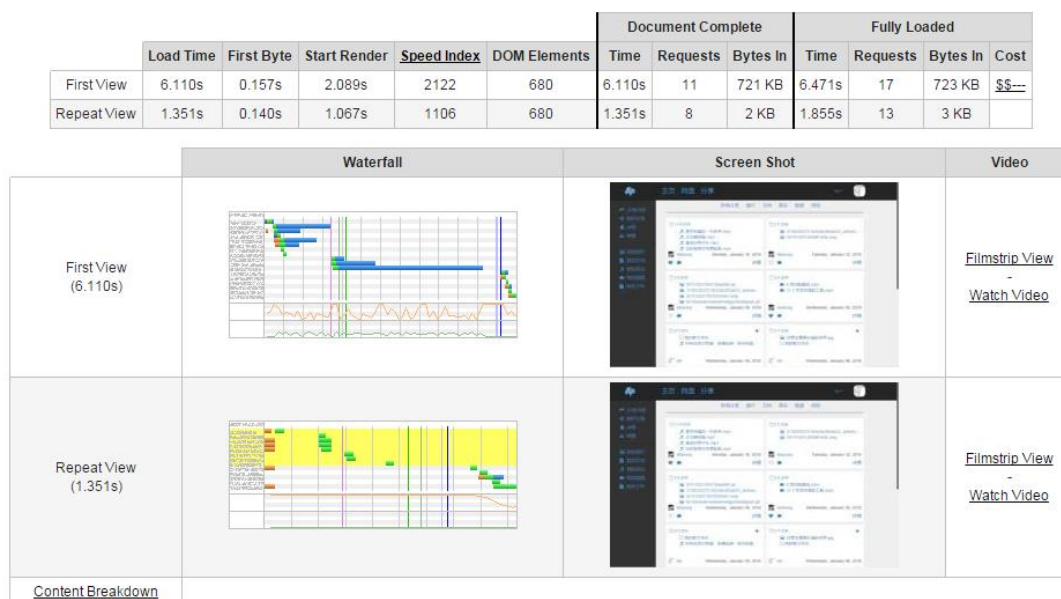


图 4-24 平台首页加载详细数据图

在第一次加载过程中，所有内容的加载完成的时间为 6.110s，而在重复加载中只需 1.351s。上图还提供了请求数、页面渲染起始点等参数。

(2) 网络负载性能测试

LoadImpact 是开源的网络负载量测试工具，可以有效地测试服务器负载性能，模拟不同数量的在线用户，测试在不同用户量的情况下网站的响应时间。并以此估算服务器的负载量。

本次测试中，使用的连接点位于新加坡，文件管理平台服务器位于中国杭州，测试的负载连接数从 0 到 50 进行持续模拟。在 5 分钟内的负载性能测试中，LoadImpact 记录了活跃测试连接数、加载时间以及带宽等数据信息，根据这个数据可以分析网站在不同用户量下的负载性能。因为网站部署服务器的最大带宽为 1Mbps，因此在测试过程中的速度基本在 1Mbps 上下。活动的连接数从 0 逐渐增加到 50 过程中，连接的加载时间在不断增加，在活动连接达到峰值 50 时，加载时间也到达顶点 45 秒。因为平台的最大带宽目前在 1Mbps，而且模拟测试的节点位于新加坡，所以在测试过程中，当 active VUs 达到一定量时，平台的负载性能会有很大影响。通过测试，当 active VUs 在 30 以下是，加载时间位于 7 秒左右，而高于 30 时，加载时间急剧上升，因此合适的负载量在 30 左右。其详细的负载性能测试如图 4-25 所示：

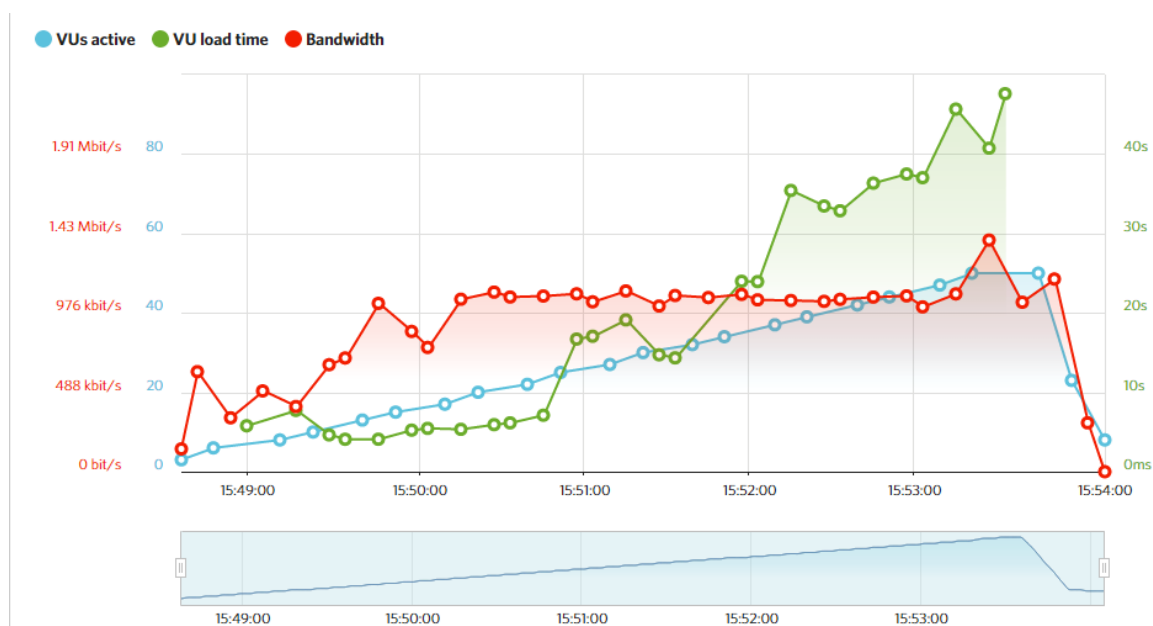


图 4-25 平台负载性能测试图

4.5 本章小结

本章通过对重要的功能模块的各个功能实现的详细描述，结合时序图或流程图具体讲解了每个功能的逻辑实现，最后还提供了各功能模块的实现效果图。本章着重实现了文件管理平台的文件管理模块、社交互动模块以及社区资源互助功能。文件管理模块包括文件的上传、下载、删除以及文件夹管理等；社交互动模块主要介绍了分享内容的点赞、评论、用户关注以及消息分发功能；社区资源互助功能为用户提供了一个互帮互助的渠道。本章最后就平台整体结构进行了详细功能测试以及性能测试。

5 总结与展望

5.1 论文工作总结

本文从现实的需要开始分析,通过国内外主流的在线文件存储管理平台的探究、对比与分析,在分析对比过程中,找到了本论文重点的工作内容。确定了本论文的重点内容后,从用户需求分析着手,罗列出本平台需要设计的重点,以及平台的大致的框架结构;从功能需求分析中,将平台所需要实现的各个功能模块分离出来,每个模块又对应着各个子功能,通过分析这些子功能,逐渐完成平台的整体设计。

通过详细的需求分析后,确定了各大功能模块,分别是用户模块、文件模块、社交模块以及社区资源共享功能。根据各个功能模块所要实现的功能逻辑构建文件管理平台的基础结构,包括 Web 服务器的搭建、静态资源服务的存储、数据库管理系统的设计以及 Web 前端功能逻辑设计。

从平台的基础结构上,探究平台所需要的技术知识结构,基于 NodeJS 搭建了一个 Web 服务器,利用七牛云存储完成平台静态资源的存储与基本的数据处理,使用 MySQL 管理平台上需要的各类数据,利用 RequireJS 构建前端的模块化管理等。

确定实现平台所需要的技术知识后,分模块实现平台的各个功能。平台整体实现完成后,再对平台的各功能进行测试,并使用 WebPageTest 测试平台的性能,确保平台可以长时间高效无误的运行。

本文中实现的主要功能包括文件的管理、社交互动以及社区资源互助。其中文件管理模块中包括文件夹的管理、文件的上传删除、文件的下载以及基于 HTML5 的在线预览功能。社交模块包括用户之间的相互关注、分享内容评论、点赞等功能。社区资源互助提供在线请求资源与资源共享功能,用户可以根据需求在平台上发布请求帖子,其他用户可以帮助提供资源,构建一个互帮互助的社区空间。文件管理平台上的子功能都是相互独立,逻辑单一的,可以很好地实现可拔插复用。利用本系统提供的功能,用户可以将本地的内容线上保存、在线浏览。利用这种方式可以更加有效、安全的保存、预览各类文件,降低了本地文件的安全性隐患,消除了跨设备访问的瓶颈以及各类文件预览方式的复杂性。

5.2 未来工作展望

本文研究的是一个基于云存储的在线文件管理平台，利用这个平台可以在线存储文件、在线文件的预览、文件的管理等，而且平台利用 WebSocket 实时双向通信功能提供了用户社交以及实时消息处理功能。但是文件管理平台功能还需要一步扩展，而且目前平台还有些许不足。

首先在文件预览方面，文件格式有太多种类型，而平台上支持的文件预览格式种类比较有限。尤其是音视频这类多媒体文件的预览，因为采用 HTML5 多媒体播放接口，摒弃了引用第三方插件播放形式，因此这极大的影响了平台对多媒体多类型文件的支持。在 HTML5 的多媒体播放下，音频文件暂时只支持 WAV、Ogg、MP3 这三种格式，而视频的播放只能支持 Ogg、MPEG4、WebM 这三类格式的文件。所以在平台改进的过程中，提供对各类流行文件格式的在线预览功能是关键。庆幸的是，七牛云存储服务提供的音视频格式转换，可以将上传的不兼容多媒体文件转换为 HTML5 播放器支持的文件格式，这将极大的扩展平台的实用性。

其次，平台目前不能对上传文件进行内容识别，这在资源涉黄的检测上带来困难，而且不能阻止各类盗版文件在平台上分享。

最后，社交平台上的功能还是有待扩展的。比如用户推荐功能、分享文件推送功能等。平台可以根据用户的活跃程度确定用户的等级，将优秀的用户推荐给平台上更多的用户认识；平台利用分享文件的浏览量、关注度等属性来实现对优质分享的推荐；而且目前平台主要适用于 PC 端，如何兼容手机浏览器，实现跨终端功能也是今后要考虑的。

总的来说，文件管理平台虽然已经在线上可以长时间高效的运行，但是文件管理平台还有很大的发展空间，利用平台良好的可扩展性实现对平台其他功能的集成是平台随后需要做的工作。

致谢

岁月匆匆，如白驹过隙。在华中科技大学不知不觉中已经学习了六年，本科四年的积累，硕士两年的升华，这其中经历很多也学习到了很多。自己也逐渐从一个无知懵懂的 Web 前端编程新手成长为可以自主实现一个平台的熟练者。

华科的种种依旧历历在目，春日的玉兰芬芳，夏日的荷塘月色，秋日的丹桂飘香，冬日的和煦阳光，还有挥斥方遒的书生意气。所有的所有构成了华科这个历史悠久学术氛围浓厚，而不失美丽的大学。在这里的六年时光里，有同学的指点，朋友的帮助，还要循循善诱的老师，伴我成长，助我向前。

在即将毕业的此刻，我首先要感谢我的导师，夏天教授。一位年轻儒雅的导师，不仅在生活给予照顾，更是学习工作中的引路人。大学之道，在明明德。实验室的许炜老师就是那个让我在生活学习中明德行，通事理的指导者。感谢导师们的倾囊相授，无私奉献。

与此同时，我还要感谢在研究生期间一直帮助我、指导我 Web 前端开发的师兄林寿山，一位才识渊博、无私好学、乐于分享的典范。感谢赵先阳学长在 Web 后端开发中的指导，让我的编程知识更加广泛。感谢在实验室学习期间的所有的同伴们，因为有你们我不会太孤单。我还要感谢我的室友，我的好朋友张建佳、文念，感谢你们一起伴我走过快乐时光、失意岁月，一起把酒言欢、倾诉衷肠，不离不弃。愿时光虽逝，情谊永存。

参考文献

- [1] Tu, Min, Zhu, et al. Application and Research of HTML5 Representing future WEB Developed Standard[C]// 2011. 2011.
- [2] Hale J S. Amazon Cloud Drive forensic analysis[J]. Digital Investigation, 2013, 10(3):259-265.
- [3] <https://www.google.com/drive/>
- [4] <https://onedrive.live.com/>
- [5] Tilkov S, Vinoski S. Node. js: Using javascript to build high-performance network programs[J]. IEEE Internet Computing, 2010, 14(6): 80.
- [6] 任中方, 张华, 闫明松,等. MVC 模式研究的综述[J]. 计算机应用研究, 2004, 21(10):1-4.
- [7] Mardan A. Pro Express. js: Master Express. js: The Node. js Framework For Your Web Development[M]. Apress, 2014.
- [8] 许会元, 何利力. NodeJS 的异步非阻塞 I/O 研究[J]. 工业控制计算机, 2015, 28(3):127-129.
- [9] Fielding R, Gettys J, Mogul J, et al. Hypertext transfer protocol--HTTP/1.1[J]. 1999.
- [10] Graham I S. The HTML sourcebook[M]. John Wiley & Sons, Inc., 1995.
- [11] MySQL A B. MySQL[J]. 2001.
- [12] Sanfilippo S, Noordhuis P. Redis[J]. 2011-03-19]. <http://redis.io>, 2009.
- [13] 七牛云存储. 云存储行业呈现新特征,将成公有云发展核心动力[J]. 电脑与电信, 2016(1).
- [14] Tiwari D, Solihin Y. Architectural characterization and similarity analysis of sunspider and Google's V8 Javascript benchmarks[C]//Performance Analysis of Systems and Software (ISPASS), 2012 IEEE International Symposium on. IEEE, 2012: 221-232.
- [15] Fielding R. Representational state transfer[J]. Architectural Styles and the Design of Network-based Software Architecture, 2000: 76-85.

- [16] Han J, Haihong E, Le G, et al. Survey on NoSQL database[C]//Pervasive computing and applications (ICPCA), 2011 6th international conference on. IEEE, 2011: 363-366.
- [17] Carlson J L. Redis in Action[M]. Manning Publications Co., 2013.
- [18] <https://www.navicat.com/download/navicat-for-mysql>
- [19] Wu J, Ping L, Ge X, et al. Cloud Storage as the Infrastructure of Cloud Computing[C]// Proceedings of 2010 International Conference on Intelligent Computing and Cognitive Informatics (ICICCI 2010). 2010:380-383.
- [20] 张辰, 郭中豪, 张勐. 浅谈云计算与云存储[C]// 第二十八届中国(天津)2014IT、网络、信息技术、电子、仪器仪表创新学术会议论文集. 2014.
- [21] 周可, 王桦, 李春花. 云存储技术及其应用[J]. 中兴通讯技术, 2010, 16(4):24-27.
- [22] 张龙立. 云存储技术探讨[J]. 电信科学, 2010(S1):71-74.
- [23] <http://expressjs.com/>
- [24] 沈珏. MVC 设计模式[J]. 科技广场, 2009 (9): 249-250.
- [25] Li Y, Cui D. Improvement and application of MVC design patterns [J]. Computer Engineering, 2005, 9: 35.
- [26] 侯静, 欧阳荣彬, 种连荣, 等. 基于 Web 标准的高校网站建设与实现[J]. 中山大学学报: 自然科学版, 2009, 48(3): 91-99.
- [27] 马新强, 孙兆, 袁哲, 等. Web 标准与 HTML5 的核心技术研究[J]. 重庆文理学院学报:自然科学版, 2010, 29(6):61-64.
- [28] Bray T, Paoli J, Sperberg-McQueen C M, et al. Extensible markup language (XML)[J]. World Wide Web Consortium Recommendation REC-xml-19980210. <http://www.w3.org/TR/1998/REC-xml-19980210>, 1998, 16.
- [29] Briggs O, Champeon S, Costello E, et al. Cascading Style Sheets[M]. 2004.
- [30] ECMAScript E, European Computer Manufacturers Association. ECMAScript Language Specification[J]. 2011.
- [31] Chakrabarti S. Integrating the document object model with hyperlinks for enhanced topic distillation and information extraction[C]//Proceedings of the 10th international conference on World Wide Web. ACM, 2001: 211-220.

- [32] Crane D, Pascarello E, James D. Ajax in Action[J]. Citeseer, 2005.
- [33] Yang J, Liao Z W, Liu F. The impact of Ajax on network performance[J]. Journal of China Universities of Posts & Telecommunications, 2007, 14:32-34.
- [34] Fette I, Melnikov A. The websocket protocol[J]. 2011.
- [35] Dai li L I, Chen R. The Research of WebSocket Based on Web Real-time Communication[J]. Computer Knowledge & Technology, 2010.
- [36] 林瑶. web 实时数据同步研究[D]. 北京邮电大学, 2015.
- [37] Rauch G. Socket. IO-the cross-browser WebSocket for realtime apps[J]. 2013.
- [38] Catchpole L, Vossen J, Briski E, et al. Web session collaboration: U.S. Patent Application 10/002,662[P]. 2001-11-30.
- [39] Postel J. Simple mail transfer protocol[J]. Information Sciences, 1982.
- [40] 张文东. 基于 Web 的安全电子邮件系统的研究与实现[D]. 新疆大学, 2005.
- [41] 刘海, 徐芳, 郭帆. 防范 XSS 攻击的研究综述[J]. 计算机与现代化, 2011 (8): 174-178.
- [42] 吴耀斌, 王科, 龙岳红. 基于跨站脚本的网络漏洞攻击与防范[J]. 计算机系统应用, 2008, 17(1): 38-40.
- [43] <http://www.webpagetest.org/>
- [44] Viscomi R, Duran M, Davies A. Using WebPageTest[J]. 2015.